

TEST DRIVING RAILS

**TAKING MINITEST AND
FIXTURES FOR A SPIN**



1st edition

By Josef Strzibny

Test Driving Rails

Taking Minitest and Fixtures for a spin

Josef Strzibny

Version 1.0

Table of Contents

Foreword	2
1. Introduction	5
2. Minitest	6
2.1. Tests and Specs	6
2.2. Runner	8
2.3. Reporters	12
2.4. Assertions	13
3. Doubles	22
3.1. Mocks	22
3.2. Stubs	24
3.3. Mocha	28
3.4. Constants	30
3.5. Environment	31
4. Testing Rails	32
5. Fixtures	36
5.1. Active Record	36
5.2. Active Storage	45
5.3. Action Text	47
5.4. Helpers	48
6. Model tests	50
6.1. Callbacks	53
6.2. Concerns	53
6.3. Attachments	55
6.4. Queues and Jobs	55
6.5. Action Cable	57
6.6. Emails	59
6.7. Services	61
6.8. Time	61
6.9. HTTP	61
7. Controller tests	66
7.1. Actions	66
7.2. Responses	69
7.3. Cookies	71
7.4. Pagination	73
7.5. Attachments	74
7.6. Translations	74

8. System tests	75
8.1. Configuration	75
8.2. Actions	77
8.3. Finders	79
8.4. Matchers	80
8.5. Sessions	82
8.6. Timeouts	84
8.7. Screenshots	86
9. Continuous Integration	88
10. Reference	92
10.1. Finders	92
10.2. Matchers	94
11. Fin	101

Rails blends Minitest and fixtures together in a seamless integration that gives developers testing super powers out-of-the-box. This guidebook is my opinionated take on testing Ruby on Rails applications with these defaults.

NOTE | Rails version used in this revision is 8.0.

@ Josef Strzibny

I would kindly ask you to not distribute this book or any of its parts without my permission. Feel free to reach me for any kind of questions at strzibny@strzibny.name or at <https://x.com/strzibnyj>.

Foreword

I met Josef on social media because I was so pleased with Kamal Handbook and wanted to let him know. Kamal Handbook is the de facto guide on deploying applications with this new tool, and it's absolutely terrific. You should read it.

This is why I was excited to learn that he was writing about one of my pet topics. I hoped that it would be just as good as his previous work. I was not disappointed. You are about to enjoy what I consider to be the de facto guide on this topic.

When Ruby on Rails arrived on the scene more than two decades ago, it changed the entire landscape of web application development. Seemingly overnight. Developing web applications was a very different beast before Rails, and has been changed for the better.

Ruby on Rails is now geriatric by "modern" software standards. It has proven itself to be considerably reliable, resilient, and even scalable. Despite its resilience and longevity, there will be a day in the future when Rails is a distant memory for only the most retired among us.

The things we learned from Rails will either be so obvious that people will be surprised that there was once a time when they weren't standard, or they'll be nearly entirely forgotten in history for future generations to re-invent from first principles over and over.

If we take only one lesson from our time with Ruby on Rails while forgetting everything else that Rails brought to our lives, I hope it's *convention over configuration*.

Convention over configuration is the simple but powerful idea that by embracing sensible defaults, avoiding surprises, and making the easy things free, we can move forward rapidly. By being freed from the shackles of endless configuration and special cases, we can focus on the things that make our software unique and bring our users joy.

One of those sensible defaults that Rails provides is in-built testing support,

with tooling and helpers to make automated testing easy and, in typical Ruby fashion, fun! We now take it for granted in the Ruby universe that the libraries and frameworks we use have tools and helpers to make testing easier, and I think this in no small way thanks to Rails showing us the way.

As a long-time practitioner and teacher of automated testing in general, and Test-Driven Development in particular, I was overjoyed when I first typed `rails new` and saw that empty test directory staring me in the face. I had a warm feeling knowing that testing was taken seriously, and treated as important by the framework and the folks that created it. I knew that I'd be able to develop robust applications with confidence.

To those who had never been exposed to automated testing, that test directory raised questions. "What goes here?" leads to "Why would I want tests?" which leads to "How do I write better tests?"

Testing support is a *sensible default*. This was plainly obvious to me at the time, but it's a lesson that a lot of folks in software learned thanks to Rails.

While supporting automated testing is necessary, it isn't sufficient. Testing is a skill to be learned and mastered. The patterns and tools are most effective when they're well-understood and used effectively.

Test Driving Rails is a concise yet incredibly complete guide to the patterns and tools we have available in Rails to test our applications. The knowledge contained in this book will ensure you have the knowledge and skills you need to ensure that your software does exactly what you said it would, every time you make a change.

It has been my absolute pleasure to read this book, and I am happy to report that it has earned an important place in my toolbox. If a book on a thing I am an expert in can earn a permanent place in my toolbox, I look forward to what it can do in yours.

Ruby on Rails provides everything you need to write effective tests. There's no need for fancy DSLs or obscure Ruby features. You just need to use what's already there. This book will show you how to make the best use of what you already have.

I hope you enjoy this book as much as I do, and that you'll use it to develop better applications, as I will.

Happy Testing!

Steven R. Baker

— Steven R. Baker, the original author of minitest/mock and RSpec

Chapter 1. Introduction

Have you ever worked on completely untested applications? It can be incredibly painful. Without automated tests, it's hard to ensure stability with code changes, protect production data, minimize the impact of bugs, and reduce the time needed for manual testing.

Automated testing gives us a fast development and deployment cycle. It gives us the confidence to ship more in a shorter amount of time. Luckily, Rails provides us with seamless Minitest integration and a way to create test data with fixtures.

Despite Minitest prominence in Ruby, Rails, and eco-system libraries, the industry overwhelmingly adopted RSpec for testing. And I felt the art for default Rails tests was lost.

I have stayed true and loyal to Minitest for all of my projects over the years. Minitest is an important item on Rails Doctrine omakase menu and I want to help bring back the spark for it with this book.

Minitest test cases are straightforward to write and more importantly effortless to read later. They are just a piece of Ruby code.

So let's go and test drive our Rails applications.

Chapter 2. Minitest

Minitest was created by Ryan Davis in 2008 as a faster and more flexible alternative to Ruby's own `Test::Unit`. Minitest replaced `Test::Unit` later in 2011 in Ruby 1.9 and it's Ruby's and Rails' default test framework. Minitest implements the [XUnit](#) pattern. Over time, more features were added, including a rich set of assertions, mocks and stubs, benchmark capabilities, or support for parallel testing. But before we have a look at all of that, let's start by looking at how to write and run basic Minitest tests.

2.1. Tests and Specs

Minitest is mostly known for simple test cases subclassed from `Minitest::Test` which provides a lightweight unit testing framework. Here's an example of a basic Minitest test:

```
require "minitest"

class LobsterRoll
  def size
    :big
  end

  def delicious?
    true
  end
end

class LobsterRollTest < Minitest::Test
  def test_is_delicious
    roll = LobsterRoll.new
    assert roll.delicious?
  end
end
```

Any method of `Minitest::Test` subclass started with `test_` is turned into runnable test cases automatically. The only other thing needed is at least one simple assertion to produce a test report.

A single test is usually composed of three steps. We prepare the test environment in a setup step before running the test case and reset the environment as part of a final teardown. The setup step prepares any data or environment needed in tests or specs. In the case of a Rails application, this also involves steps done by the Rails framework, namely loading your fixtures.

Here's an example of a `setup` step preparing an object for the test that follows:

```
class LobsterRollTest < Minitest::Test
  def setup
```

```
@roll = LobsterRoll.new
end
...
```

If you are coming from RSpec, `subject` and `let!` blocks would also be part of this simple method. There is no conditional object creation so you should include things relevant for all tests in the same class.

A test case calls the object methods under tests and the result is verified with assertions:

```
class LobsterRollTest < Minitest::Test
  ...
  def test_name
    assert @roll.delicious?
    assert_equal :size, @roll.size
  end
  ...
end
```

The simplest of assertions are `assert` and `assert_equal`. One asserts truthiness and the other equality. There are many more assertions which we'll see in the Assertions chapter, but we can test most things with these two.

Finally, there is a teardown step that should clean up the environment for the next test. This means clearing the database of the records we created or verifying mocks. If the lobster roll is an object in the database, we might want to remove it at the end:

```
require "minitest"

class LobsterRoll < ActiveRecord::Base
end

class LobsterRollTest < Minitest::Test
  def setup
    @roll = LobsterRoll.create!
  end
  ...
  def teardown
    @roll.destroy!
  end
  ...
end
```

We'll see that this is usually not needed since Rails and other Minitest libraries implement a teardown on their own. They can take advantage of Minitest lifecycle hooks to extend the tests:

```
module MyMinitestPlugin
```

```

def before_setup
  super
end

def after_setup
  super
end

def before_teardown
  super
end

def after_teardown
  super
end

class MiniTest::Test
  include MyMinitestPlugin
end

```

The above tests can also be represented with `Minitest::Spec`, a layer of syntactic sugar allowing for RSpec-like specification way of testing:

```

require "minitest/spec"

describe "LobsterRoll" do
  it "is delicious" do
    roll = LobsterRoll.new
    expect(roll.delicious?).must_equal true
  end
end

```

Similarly, Rails provides its syntactic sugar for `Minitest::Test` as we'll see later. In this book, I'll stick to the Rails style of tests, but if you like specs more, it's an option.

2.2. Runner

Once the test is written it's time to run it. Minitest suite is run with a call to `Minitest.run` which accepts list of command line arguments:

```

#!/usr/bin/env ruby

require 'minitest'

```

```
class ExampleTest < Minitest::Test
  def test_truth
    assert true
  end
end

Minitest.run(ARGV)
```

Running this test prints the following:

```
% ruby example.rb
Run options: --seed 56077

# Running:

.

Finished in 0.000289s, 3460.2454 runs/s, 3460.2454 assertions/s.

1 runs, 1 assertions, 0 failures, 0 errors, 0 skips
```

We can see one passed run with 1 assertion. And we can also see an implicit `--seed` option. Since tests are randomized by default, providing the seed makes it possible to rerun the test suite in the same order.

This brings us to `ARGV` which refers to arguments passed to the Ruby script:

```
$ ruby test.rb -n test_truth
Run options: -n test_truth --seed 33669

# Running:

.

Finished in 0.000230s, 4347.8018 runs/s, 4347.8018 assertions/s.

1 runs, 1 assertions, 0 failures, 0 errors, 0 skips
```

Now only the `test_truth` test case will be run.

We can also adjust the arguments from within the script:

```
#!/usr/bin/env ruby
```

```

require "minitest"

class ExampleTest < Minitest::Test
  def test_truth
    assert true
  end

  def test_falsehood
    refute false
  end
end

ARGV << "--verbose"

# Will only run test_truth
Minitest.run(ARGV)

# Or with fixed arguments
Minitest.run(["-n", "test_truth", "--verbose"])

```

Minitest also features a handy `autorun` method that will do the above for you using the `Kernel.at_exit` hook:

```

# test.rb
require "minitest"
...
Minitest.autorun

```

Since it's registered at the exit it's more common to see it used as `require` call at the top:

```

# test.rb
require "minitest/autorun"
...
class ExampleTest < Minitest::Test
  def test_truth
    assert true
  end
end

```

And here's the source for `Minitest.autorun` method itself:

```

# from Minitest source
def self.autorun

```

```

if Object.const_defined?(:Warning) && Warning.respond_to?(:[]=)
  Warning[:deprecated] = true
end

at_exit {
  next if $! and not ($!.kind_of? SystemExit and $!.success?)

  exit_code = nil

  pid = Process.pid
  at_exit {
    next if !Minitest.allow_fork && Process.pid != pid
    @@after_run.reverse_each(&:call)
    exit exit_code || false
  }

  exit_code = Minitest.run ARGV
} unless @@installed_at_exit
@@installed_at_exit = true
end

```

We can see that this registers Ruby's `at_exit` hook and runs `Minitest.run` in the end. But what about the `Minitest.allow_fork` check? Minitest supports forking in tests so it has to skip the `at_exit` registration in such a case.

Here's one example of a test like that:

```

require "minitest/autorun"

class ForkingTest < Minitest::Test
  def setup
    Minitest.allow_fork = true
  end

  def test_forking
    pid = fork do
      assert true
    end
    Process.wait(pid)
  end
end

```

Minitest also features built-in parallelization. To enable it we have to call `parallelize_me!` in each test class:

```
require "minitest/autorun"

class ExampleTest < Minitest::Test
  parallelize_me!

  def test_example_one
    puts Process.pid
    puts Thread.current.object_id
    assert_equal 2, 1 + 1
  end

  def test_example_two
    puts Process.pid
    puts Thread.current.object_id
    assert_equal 4, 2 + 2
  end
end
```

NOTE To set parallelization for the whole test suite at once, we can call `Minitest::Test.parallelize_me!`.

Printing `Process.pid` and `Thread.current.object_id` reveal that this parallelization is done by default using threads rather than processes. This is generally a good setting if the tests are not CPU-bound or run on a platform like JRuby.

We can control how many threads are used by setting the `MT_CPU` environment variable:

```
$ MT_CPU=4 ruby parallel_test_suite.rb
```

If we would like to parallelize the test suite using processes with plain Minitest, we have to go for a plugin such as `minitest-parallel_fork`. One good reason for that is to avoid issues with modified constants.

Rails test stack builds on top of Minitest and already comes with its options for parallelization, including process-based parallelization. We'll get back to it a bit later.

2.3. Reporters

To get the test results out of `Minitest.run`, Minitest utilizes various reporters that record single test results that can be merged into a final report.

As an example, `ProgressReporter` implements the dots you'll see on the console while running a Minitest test suite, `StatisticsReporter` handles timing and counters, and `SummaryReporter` provides the summary at the end of a test run.

We can instruct Minitest to use any different reporter with `Minitest::Reporters.use!`:

```
gem "minitest-reporters"

# test_helper.rb or spec_helper.rb
require "minitest/reporters"
Minitest::Reporters.use!(Minitest::Reporters::SpecReporter.new)
```

Suddenly the final report layout completely changes:

```
$ ruby test.rb
Started with run options -n test_truth --seed 31890

ExampleTest
  test_truth                                     PASS (0.00s)

Finished in 0.00030s
1 tests, 1 assertions, 0 failures, 0 errors, 0 skips
```

The abstract reporter interface features `start`, `record`, `report`, and `passed?` methods in case you would like to implement a custom reporter.

We don't need to configure any special reporting, but it's simply one of the many options for how to extend Minitest with plugins.

2.4. Assertions

Assertions are the bread and butter of Minitest. An assertion is a logical statement that declares something to be true and the number one assertion in Minitest is a simple method called `assert`:

```
require "minitest/autorun"

class ExampleTest < Minitest::Test
  def test_this_passes
    assert true
  end

  def test_this_fails
    assert false
  end
end
```

If an assertion fails a reporter will collect the failure and error message for later:

Run options: **--seed** 61546

Running:

.F

Finished **in** 0.000519s, 3853.5646 runs/s, 3853.5646 assertions/s.

1) Failure:

PostTest#test_this_fails [testa.rb:9]:

Expected **false** to be truthy.

2 runs, 2 assertions, 1 failures, 0 errors, 0 skips

Here's how is the assertion defined:

```
# from Minitest source
def assert test, msg = nil
  self.assertions += 1
  unless test then
    msg ||= "Expected #{mu_pp test} to be truthy."
    msg = msg.call if Proc === msg
    raise Minitest::Assertion, msg
  end
  true
end
```

You can notice that it takes an optional message. If we want we can always provide our own message for the failure:

```
require "minitest/autorun"

class ExampleTest < Minitest::Test
  def test_this_passes
    assert true, "not true"
  end

  def test_this_fails
    assert false, not true
  end
end
```

Here's the changed reporter output:

```
Run options: --seed 16569

# Running:

.F

Finished in 0.000346s, 5780.3261 runs/s, 5780.3261 assertions/s.

  1) Failure:
PostTest#test_this_fails [testa.rb:9]:
not true

2 runs, 2 assertions, 1 failures, 0 errors, 0 skips
```

This will be true for other assertions too. We can always be a bit more specific when needed.

The Minitest opposite of `assert` is `refute`. However, Rails decided to implement its own `assert_not` which is the same and based on `assert` itself:

```
# from Rails source
module ActiveSupport
  module Testing
    module Assertions
      def assert_not(object, message = nil)
        message ||= "Expected #{mu_pp(object)} to be nil or false"
        assert !object, message
      end
    end
  end
end
...
```

The `assert` assertion itself can be all you'll ever need, but there are other assertions out there. Using a more specific assertion can lead to a more useful error message or a better portrait of the semantics of the test.

2.4.1. The case of a nil

Since `nil` has a special meaning in Ruby, we get `assert_nil` and `refute_nil` in Minitest. All they do is call `nil?` on the passed object under the hood:

```
require "minitest/autorun"

class ExampleTest < Minitest::Test
  def test_nil
    assert_nil nil
  end
end
```

```
def test_not_nil
  refute_nil true
end
end
```

This check is similar to regular Ruby conditionals checking `nil` and `false` values.

You'll also notice we usually get a negative variant of the original assertion. Most of the time this is just semantics, but it might be important as we'll see with Capybara later.

2.4.2. Equality

If you aren't testing for truthiness, you are likely to test for equality with the `assert_equal` assertion. Remember that we essentially care about the value. Here's an example:

```
class Calculator
  def add(a, b)
    a + b
  end
end

require "minitest/autorun"

class CalculatorTest < Minitest::Test
  def setup
    @calculator = Calculator.new
  end

  def test_addition
    result = @calculator.add(2, 3)
    assert_equal 5, result
  end
end
```

If we dive into the source, you'll see what I mean:

```
def assert_equal exp, act, msg = nil
  msg = message(msg, E) { diff exp, act }
  result = assert exp == act, msg

  if nil == exp then
    if Minitest::VERSION =~ /^6/ then
      refute_nil exp, "Use assert_nil if expecting nil."
    else
```

```

      warn "DEPRECATED: Use assert_nil if expecting nil from #{_where}. This will fail in
Minitest 6."
    end
  end

  result
end

```

Now if need to compare the objects with Ruby's `equal?` method, you need `assert_same` which works in the same way, but will test for actual object equality in memory:

```

class Singleton
  def self.instance
    @instance ||= new
  end
end

require "minitest/autorun"

class SingletonTest < Minitest::Test
  def test_singleton_instance
    instance1 = Singleton.instance
    instance2 = Singleton.instance
    assert_same instance1, instance2
  end
end

```

Minitest also lets us to check for an object to be an instance of another with `assert_instance_of` and similarly for objects to be a kind of an object with `assert_kind_of`:

```

class User
end

class Admin < User
end

class Registration
  def initialize(admin: false)
    admin ? Admin.new : User.new
  end
end

class InstanceKindTest < Minitest::Test
  def setup
    @person = Registration.new
  end
end

```

```

    @admin = Registration.new(admin: true)
  end

  def test_create_user
    assert_instance_of User, @person
    assert_kind_of User, @person
  end

  def test_create_admin
    assert_instance_of Admin, @person
    assert_kind_of User, @person
    assert_kind_of Admin, @person
  end
end

```

2.4.3. Regexp

Equality tests aren't as useful when we don't care for an exact match. What if we have a formatter that works with dates while we only care about the correct format rather than values themselves?

We can test it using `assert_match` to match our test object against a regular expression:

```

class Message < ActiveRecord::Base
end

class Formatter
  def self.format_message(message)
    "#{message.created_at.strftime("%Y-%m-%d")}: #{message.text}"
  end
end

require "minitest/autorun"

class FormatterTest < Minitest::Test
  def setup
    message = Message.create!(text: "Hi")
  end

  def test_log
    result = Formatter.format_message(message)
    assert_match(/\d{4}-\d{2}-\d{2}: Hi/, result)
  end
end

```

This assertion will also come in handy later when matching against `response.body` in controller tests.

2.4.4. Exceptions

Sometimes things go sour and our code throws exceptions. Let's imagine a calculator that cannot accept 0 for division:

```
class Calculator
  def divide(a, b)
    raise ZeroDivisionError, "divided by 0" if b == 0
    a / b
  end
end
```

To test the raise we employ `assert_raises` assertions which take the exception class and a block:

```
require "minitest/autorun"

class CalculatorTest < Minitest::Test
  def setup
    @calculator = Calculator.new
  end

  def test_divide_by_zero
    assert_raises(ZeroDivisionError) do
      @calculator.divide(10, 0)
    end
  end
end
```

If we use `throw` for a control flow, there is also a sister assertion called `assert_throws`:

```
class CustomFlow
  def process_data(data)
    throw :invalid_data if data.nil?
    data.upcase
  end
end

require "minitest/autorun"

class CustomFlowTest < Minitest::Test
  def setup
    @custom_flow = CustomFlow.new
  end

  def test_process_data_throws_invalid_data
```

```

    assert_throws(:invalid_data) do
      @custom_flow.process_data(nil)
    end
  end
end

```

2.4.5. Output

At times we might be writing CLI programs or Rake tasks and would love to test for their output to standard output and error streams (**STDOUT** and **STDERR**). We can test that with **assert_output**.

Let's imagine we have a **Logger** class that will print standard output and error messages:

```

class Logger
  def log_info(message)
    puts "INFO: #{message}"
  end

  def log_error(message)
    warn "ERROR: #{message}"
  end
end

```

If we want to test it, we use **assert_output** with two arguments and a block. The first argument is expected **STDOUT**, the second is expected **STDERR**, and the block is our code:

```

require "minitest/autorun"

class LoggerTest < Minitest::Test
  def setup
    @logger = Logger.new
  end

  def test_log_info_outputs_to_stdout
    assert_output("INFO: Application started\n") do
      @logger.log_info("Application started")
    end
  end

  def test_log_error_outputs_to_stderr
    assert_output(nil, "ERROR: Something went wrong\n") do
      @logger.log_error("Something went wrong")
    end
  end
end

```



```
end
```

Similarly we can also expect a silence of no `STDOUT` or `STDERR` streams with `assert_silent`:

```
class SilentWorker
  def perform
    # Some operations that do not produce any output
  end
end

require "minitest/autorun"

class SilentWorkerTest < Minitest::Test
  def setup
    @worker = SilentWorker.new
  end

  def test_perform_is_silent
    assert_silence do
      @worker.perform
    end
  end
end
```

Chapter 3. Doubles

A test double is an object or a piece of code used to replace an object or code that's hard or inconvenient to test. One reason to use test doubles like mocks and stubs is to avoid slow code that's not strictly necessary to run as part of the test or to replace brittle parts that make the test unreliable (like external HTTP calls).

Test dummy is an object placeholder without any other purpose, mocks are objects that mimic and replace other internal objects while stubs isolate code we want to test from the complexities of the whole application.

Let's see how this all works in Minitest.

NOTE | Test dummies don't exist in Minitest. You pass Ruby's `Object.new` in such a case.

3.1. Mocks

Mocks are usually small handcrafted objects that are ready to be passed to methods under test to invoke a specific scenario we want to see in the test. Steven R. Baker, the original author of minitest/mock, explains mocks in relationship with object-oriented programming:

Mock objects are an often misunderstood part of testing. Possibly because they are so wonderfully simple.

Object-oriented systems are made up of *objects* which are the data, and the methods that operate on that data, and *messages* which are the way the objects communicate with each other.

When you write `receiver.do_something` it's easy to think that you're "calling" the `do_something` method on the `receiver` object. But you're actually *sending* the message `do_something` to the `receiver`.

The most common tool we use for checking things in our tests are *assertions*. Assertions are for ensuring that we have expected values in the expected places.

Mocks, on the other hand, are for making assertions on the *messages* sent to objects. Assert is for checking the state, and mocks are for verifying the interactions.

It is common to think that we use mocks to replace "expensive" or "slow" bits

with things that are faster, but that's just a side-effect. When we mock a network API call, what we're saying "It is not our job to test the network API, it is our job to ensure that we are interacting with that API in the appropriate way."

— Steven R. Baker

Technically, Minitest implements mocks in `Minitest::Mock` which lets us define expectations for called methods with `expect` and then verify them with `assert_mock` or `verify`.

Here's a small `Payment` class that can accept payments based on the provided `Provider`:

```
class StripeProvider
  def charge(user, amount)
    # code to charge customer
  end
end

class Payment
  def initialize(provider: StripeProvider.new)
    @provider = gateway
  end

  def charge(user, amount)
    @provider.charge(user, amount)
  end
end
```

If we would pass a real gateway object in the test, we would have to set up a test environment on the provider side and depend on an HTTP call. Instead, we can create a mock payment provider for the test case:

```
require "minitest/autorun"

class PaymentTest < Minitest::Test
  def setup
    @mock_provider = Minitest::Mock.new
    @mock_provider.expect(:charge, true, [Object, Numeric])
  end

  def test_user_is_charged
    payment = Payment.new(provider: @mock_provider)

    assert payment.charge(User.new, 100)
    assert_mock @mock_provider
  end
end
```

```
end  
end
```

Whenever we create a mock, we can define methods in two ways. The first one is an expectation using `expect` as in the example above:

```
@mock_provider = Minitest::Mock.new  
@mock_provider.expect(:charge, true, [Object, Numeric])
```

We pass the method name, the return value, and the method arguments at last. We can use exact values or their types for the method arguments:

```
@mock_provider = Minitest::Mock.new  
@mock_provider.expect(:charge, true, [Object, Numeric])
```

Then we call `assert_mock` which is a Minitest assertion that will run `verify` on the mock object. This method will verify that our expectations were correct.

The second option to define the mocked methods is to define them as regular Ruby object methods:

```
@mock_provider = Minitest::Mock.new  
def @mock_provider.charge(user, amount); true; end
```

We might want to define the methods without expectations if we use the same mock in many different tests.

Mocking is straightforward if it's enough to pass the right object, but most of our code might not accept dependencies this way. That's why mocks are often used together with stubs.

3.2. Stubs

Stubs are usually methods we redefine for the test under the hood. The test will run with the tested object as usual except some parts of the application are now stubbed and return whatever we define in the test. This way stubbing allows us to isolate code we are testing from other complex behavior.

Let's have a look at a similar subscription example but assume a user balance has to be adjusted when a subscription is applied:

```
class User < ActiveRecord::Base  
  attr_writer :balance  
end
```

```

class StripeProvider
  def charge(user, amount)
    # Some Stripe calls
  end
end

class Subscription
  def self.charge(user, amount)
    provider = StripeProvider.new
    provider.charge(user, amount)
    user.balance = (user.balance - amount)
    user.save!
  end
end

```

Note that we call `charge` to charge the user on the provider side but also adjust the local account balance.

We still need to provide the test with a mock provider. Only this time we don't have a good place to do it since it never comes out as an argument. This is when stubs come in since they can replace both class and object methods.

To sneak in the mocked provider we'll stub the `new` method on the `StripeProvider`:

```

class SubscriptionTest < Minitest::Test
  def setup
    @mock_provider = Minitest::Mock.new
    @mock_provider.expect(:charge, true, [Object, Numeric])
  end

  def test_charge_adjusts_balance
    StripeProvider.stub :new, @mock_provider do
      user = User.new(balance: 300)
      assert Subscription.charge(user, 100)
      assert_equal 200, user.reload.balance
    end
    assert_mock @mock_provider
  end
end

```

Stubs in Minitest takes the method name, return value, and a block in which they operate. Here we replaced the regular initialization with a mocked object that could later call `charge` without any external calls.

If we need to stub more things at once, we need to nest the blocks together. Let's say we want to save the time of the transaction:

```

class Subscription
  def self.charge(user, amount)
    provider = StripeProvider.new
    provider.charge(user, amount)
    user.balance = (user.balance - amount)
    user.last_transation_at = Time.now
    user.save!
  end
end

```

Then we need to add the extra `stub` call as another nested block:

```

class SubscriptionTest < Minitest::Test
  def setup
    @mock_provider = Minitest::Mock.new
    @mock_provider.expect(:charge, true, [Object, Numeric])
  end

  def test_charge_adjusts_balance
    StripeProvider.stub :new, @mock_provider do
      Time.stub(:now, Time.new(2024, 1, 1)) do
        user = User.new(balance: 300)
        assert Subscription.charge(user, 100)
        assert_equal 200, user.reload.balance
        assert_equal Time.new(2024, 1, 1).to_i, user.last_transation_at.to_i
      end
    end
    assert_mock @mock_provider
  end
end

```

The cumbersome multi-block nature of stubbing more constants is by design. If it doesn't look good, the code under test should be likely refactored.

We can stub objects in a similar fashion. In fact we can even go one step further and stub their instance variables' methods. Let's say we'll process orders with our `StripeProcessor` based on the result of the payment:

```

class StripeProvider
  def charge(user, amount)
    # Some Stripe calls
  end
end

```

```

class OrderProcessor
  def initialize(user, amount)
    @user = user
    @amount = amount
    @stripe_provider = StripeProvider.new
  end

  def process_order
    if @stripe_provider.charge(@user, @amount)
      "Payment successful"
    else
      "Payment failed"
    end
  end
end
end

```

Then by creating `@order_processor` object beforehand we can access its instance variable `@stripe_provider` with `instance_variable_get` and stub their methods:

```

class OrderProcessorTest < Minitest::Test
  def setup
    @user = User.new
    @amount = 100
    @order_processor = OrderProcessor.new(@user, @amount)
  end

  def test_process_order_success
    @order_processor.instance_variable_get(:@stripe_provider)
      .stub(:charge, true) do
      result = @order_processor.process_order
      assert_equal "Payment successful", result
    end
  end

  def test_process_order_failure
    @order_processor.instance_variable_get(:@stripe_provider)
      .stub(:charge, false) do
      result = @order_processor.process_order
      assert_equal "Payment failed", result
    end
  end
end

```

This is quite powerful, but it might also be a sign that your tests are getting coupled with the implementation.

3.3. Mocha

If you are coming from other testing libraries you might find the syntax of Minitest mocks and stubs a bit unusual. Luckily we can add a testing library called Mocha to the mix to get a little bit smoother experience.

IMPORTANT

Mocha is not thread-safe. This is usually not a big problem since tests in Rails are parallelized with processes but could be a problem while testing threaded code.

To add Mocha, we add it as a dependency and require `mocha/minitest` at the bottom of the test helper:

```
# Gemfile in Rails app
gem "mocha"

# At bottom of test/test_helper.rb
require "mocha/minitest"
```

So what do Mocha syntax improvements give us? When stubbing an object, we can now use `with` for arguments and `returns` for return values instead of blocks. Let's see what this looks like for a generic mock object:

```
# Create a mock
mocked_provider = mock()

# Or even give it a name
mocked_provider = mock("StripeProvider")

# Provide a return value for specific arguments
mocked_provider.expects(:charge).with(user, "300").returns(true)
```

The mock object will search through its expectations from newest to oldest to find the match. We can use `never`, `once`, `at_least_once`, `at_least(number)`, or `at_most(number)` at the end to check the number of invocations:

```
# expect exactly one charge
mocked_provider.expects(:charge).with(user, "300").returns(true).once
```

These kind of expectations will be checked at the end of the test automatically.

Mocha also allows us to stub any of our objects. It works the same way:


```
charge = Charge.new
charge.expects(:provider).returns(:stripe)
```

Only now we can also stub class methods and any instances of said class.

And how to do the same for a class method:

```
charge = Charge.new
Charge.stubs(:find).with(1).returns(charge)
```

Here's how to stub all instance methods of a class:

```
Charge.any_instance.stubs(:allowed?).returns(true)
```

If we wanted, we could replace `stubs` with the same `expects` call as before if we care about the method to be called during the test. The use of these methods should depend on our intent, but they are the same thing from the Mocha point of view.

If we need to raise an exception or throw instead of providing a standard return value, we can do it with `raises` and `throws`:

```
mock_provider = mock()
mock_provider.stubs(:transfer).raises(ProviderError, "Insufficient funds")
mock_provider.stubs(:method_with_exception).raises(ProviderError.new("Insufficient funds"))
mock_provider.stubs(:transfer).throws(:done)
```

Finally Mocha also allows us to chain what should happen at each invocation:

```
mock_provider = mock()
mock_provider.stubs(:transfer).returns(:ok).then.returns(:ok).then.raises(ProviderError, "Insufficient funds")

mock_provider.transfer # => :ok
mock_provider.transfer # => :ok
mock_provider.transfer # => raises exception
```

If we return to the previous example combining mocks and stubs, here's how it could look like with Mocha:

```
class SubscriptionTest < Minitest::Test
```

```

def setup
  mocked_provider = mock("StripeProvider")
  mocked_provider.expects(:charge).with(user, "300").returns(true)
  StripeProvider.any_instance.stubs(:provider).returns(mock_provider)
  User.any_instance.stub(:verified?).returns(true)
end

def test_charge_adjusts_balance_for_verified user
  user = User.new(balance: 100)
  assert Subscription.charge(user, 300) assert_equal user.reload.balance, 200
end
end

```

3.4. Constants

Minitest doesn't come with a specific feature to change constants. Changing a constant value changes it for all threads inside a Ruby process and so we should try to avoid it if we can.

However, if we need to change a value of a constant under test, we can do it with the `minitest-stub-const` plugin that adds basic support for stubbing constant:

```

# Gemfile
gem "minitest-stub-const", only: :test

```

Let's say we have some fixed configuration defined in a couple of constants:

```

module Configuration
  MAX_ATTEMPTS = 5
  TIMEOUT = 30
  API_ENDPOINT = "https://api.example.com"
end

```

It might be difficult to efficiently test code that relies on this constants. We might find a way how to circumvent the issue, but what if we want to temporarily change the value in a test?

```

...
require "minitest/autorun"

class ConfigurationTest < Minitest::Test
  def test_stub_constants
    verbose = $VERBOSE
    $VERBOSE = nil
    original_value = Configuration::API_ENDPOINT

```

```

Configuration.const_set(:API_ENDPOINT, "https://changed.example.com")
assert_equal "https://changed.example.com", Configuration::API_ENDPOINT
ensure
  Configuration.const_set(:API_ENDPOINT, original_value)
  $VERBOSE = verbose
end
end

```

It's important to set the value of the constant back in Ruby's `ensure` block since the test could be failing. And since changing a constant in Ruby issues a warning, we temporarily set `$VERBOSE` env to `nil` to suppress it.

But what about the other threads we mentioned in the beginning?

If we parallelize tests with threads rather than processes (with `parallelize_me!`), two competing tests for a constant will fail one of the tests. You can try it yourself by creating another test while adding some `sleep` to one of them before the value is changed back.

So that's already two good reasons to avoid stubbing constants and treating this kind of test as a last resort.

3.5. Environment

Similarly to constants we might need to control the settings of environment variables. This can be solved with a gem called `ClimateControl`:

```

# Gemfile
gem "climate_control", only: :test

```

`ClimateControl.modify` takes the list of envs to change and a block that will run the code with the altered environment:

```

...
require "minitest/autorun"
require "climate_control"

class EnvironmentTest < Minitest::Test
  def test_stub_env
    ClimateControl.modify FOO: "bar", MAIL_FROM: "us@example.com" do
      assert_equal "bar", ENV["FOO"]
      assert_equal "us@example.com", ENV["MAIL_FROM"]
    end
  end
end

```

Chapter 4. Testing Rails

Rails enhances the basic Minitest tests with its own syntax, assertions, and capabilities. Similarly, Rails tasks wrap the Minitest runner and give us a bit more options on how to run the test suite. The framework also comes with its way of creating test data called fixtures.

Rails splits tests into three different types, `ActiveSupport::TestCase` is the basic enhancement to `Minitest::Test` while `ActionDispatch::IntegrationTest` and `ActionDispatch::SystemTestCase` lets us efficiently test the higher level of the application stack. We'll see them all in detail in the following chapters.

When you generate a brand new Rails application, you should get a basic test structure under `test` which also comes with `test_helper.rb` and `application_system_test_case.rb` files:

```
test/fixtures
test/models
test/mailers
test/controllers
test/integration
test/channels
test/system
test/test_helper.rb
test/application_system_test_case.rb
```

These configuration files drive the configuration of the test suite. Every test file will depend on `test_helper.rb`:

```
# test/test_helper.rb
ENV["RAILS_ENV"] ||= "test"
require_relative "../config/environment"
require "rails/test_help"

module ActiveSupport
  class TestCase
    # Run tests in parallel with specified workers
    parallelize(workers: :number_of_processors)

    # Setup all fixtures in test/fixtures/*.yml for all tests in alphabetical order.
    fixtures :all

    # Add more helper methods to be used by all tests here...
  end
end
```

As you can see the initial scaffolding sets the tests to be run in parallel using processes and the number of processes is determined based on available processor cores. As we discussed before this is a safer setting than relying on threads for most people.

`ActiveSupport::TestCase` is also there for easy extension on our part. Adding a method here would make it available throughout the test suite. And while system tests also depend on these settings, they are a little bit special and come with their own `ApplicationSystemTestCase`:

```
# test/application_system_test_case.rb
require "test_helper"

class ApplicationSystemTestCase < ActionDispatch::SystemTestCase
  driven_by :selenium, using: :chrome, screen_size: [1400, 1400]
end
```

The basic settings set up a browser driver and a screen size. We'll get back to it in the System tests chapter.

Since the project structure starts empty of any test cases you can run a Rails scaffold generator to add some tests:

```
$ bin/rails g scaffold Brand name:string
  invoke  active_record
  create   db/migrate/20240827030639_create_brands.rb
  create   app/models/brand.rb
  invoke  test_unit
  create   test/models/brand_test.rb
  create   test/fixtures/cars.yml
  create   test/fixtures/brands.yml
  create   test/system/brands_test.rb
  invoke  scaffold_controller
  invoke  test_unit
  create   test/controllers/brands_controller_test.rb
  create   test/system/brands_test.rb
  ...
$ bin/rails g scaffold Car name:string brand:references
...
```

Rails scaffolding adds relevant test files by default. Similarly generating an individual model creates a related test case with a model fixture file and generating a controller creates a controller test file.

We can also ask Rails to generate new test files directly:

```
$ rails generate test_unit:model Car
```

```

    create test/models/car_test.rb
    create test/fixtures/cars.yml
$ rails generate test_unit:controller Car
    create test/controllers/cars_controller_test.rb
$ rails generate test_unit:system Car
    create test/system/cars_test.rb

```

Most tests in Rails look pretty similar to what we have seen so far with plain Minitest, but there is a specific syntax for writing tests:

```

require "test_helper"

class CarTest < ActiveSupport::TestCase
  setup do
    @car = cars(:vw_golf)
  end

  test ".automatic_transmission" do
    assert_equal 3, Car.automatic_transmission.count
  end

  test "#full_name" do
    assert_equal "VW Golf", @car.full_name
  end
end

```

Test methods get a simple DSL to define the test method with a string. Technically speaking the description gets translated to a usual snake case method name for the test method:

```

# ActiveSupport/lib/active_support/testing/declarative.rb

module ActiveSupport
  module Testing
    module Declarative
      unless defined?(Spec)
        # Helper to define a test method using a String. Under the hood, it replaces
        # spaces with underscores and defines the test method.
        #
        #   test "verify something" do
        #     ...
        #   end
        def test(name, &block)
          test_name = "test_#{name.gsub(/\s+/, '_')}.to_sym"
          defined = method_defined? test_name
          raise "#{test_name} is already defined in #{self}" if defined

```

```
    if block_given?  
      define_method(test_name, &block)  
    else  
      define_method(test_name) do  
        flunk "No implementation provided for #{name}"  
      end  
    end  
  end  
end  
end  
end  
end  
end
```

Similarly, `setup` and `teardown` get easy-to-read block syntax instead of the usual method definition. I'll use this default syntax from now on.

Once we have some tests, we can run them. Running the whole test suite is done with `rails tests:all`:

```
$ bin/rails tests:all
```

The `rails tests` task alone skips system tests to allow for faster feedback.

Chapter 5. Fixtures

Test data are an essential part of most tests. Apart from creating test data on spot inside a test case, we might want to reference previously prepared test data called fixtures or implement a factory objects able to create such data whenever needed.

Fixtures in Rails are predefined sample data one can use later in the test suite. You might have noticed that newly generated Rails applications feature YAML files in the `test/fixtures` directory representing our models. They are the most common fixtures, but not the only ones.

The way I think about fixtures is that they are a small representation of your application world. They are a small snapshot of a running application at a specific time. We design them to cover 60-80% cases while keeping the amount of fixtures small.

Let's start by looking more closely on fixtures representing Active Record objects.

5.1. Active Record

Rails fixtures representing database-backed models are defined as part of the Active Record framework and loaded automatically from the `ActiveSupport::TestCase.fixture_paths` locations. They are defined in the form of YAML files where each file represents one kind of AR model.

Let's say we want to have fixtures for the `User` model. Then we create a related file inside one of the `fixture_paths` locations called `users.yml`, by default in `test/fixtures`. A small example of the `users.yml` file might look like this:

```
# test/fixtures/users.yml
joe:
  name: Joe
```

At a minimum, we have to provide all required attributes apart from the record ID. If they should represent a different model then implied by the path, we can change it by providing a specific `model_class`:

```
# test/fixtures/users.yml
_fixture:
  model_class: AdminUser
joe:
  name: Joe
```

Loading and managing these fixtures in Rails is handled by the `FixtureSet` class. We have already seen that the test helper loads all fixtures:


```
# test/test_helper.rb
...

module ActiveSupport
  class TestCase
    ...
    fixtures :all
  end
end
```

We could also load only users fixtures for a user test:

```
# test/test_helper.rb
...

class UserTest < ActiveSupport::TestCase
  fixtures :users

  test "#name" do
    assert users(:joe).name, "Joe"
  end
end
```

You can notice we also have a useful helper `users` (named after the model) to reference a particular fixture by name. It returns the associated Active Record object or an array if there are more results. If we used a namespaced model like `Admin::User`, fixtures would go in `test/fixtures/admin/users.yml` and would be referenced similarly:

```
# test/test_helper.rb
...

class UserTest < ActiveSupport::TestCase
  fixtures :admin/users

  test "#name" do
    assert admin_users(:joe).name, "Joe"
  end
end
```

Stating what fixtures you need will automatically load the data for every test. If we want to load fixtures elsewhere, we can do it ourselves by calling the Rake task or `ActiveRecord::FixtureSet.create_fixtures`:

```
# db/seeds.rb
unless Rails.env.production?
```

```
# Load all fixtures
Rake.application["db:fixtures:load"].invoke

# Load only topics table
ActiveRecord::FixtureSet.create_fixtures("test/fixtures", "topics")
end
```

This can come in handy in a seed file where we can reuse our hand-crafted fixtures for development dummy data or even bootstrap structural data in production.

Now let's imagine a more realistic example:

```
# test/fixtures/users.yml
<% password = "test123456" %>

joe:
  name: Joe
  email: <%= Faker::Internet.unique.email %>
  encrypted_password: <%= Devise::Encryptor.digest(User, password) %>
  confirmed_at: <%= DateTime.now %>
  time_zone: "Berlin"

chris:
  ...
  ...
```

Here are some first things to notice:

- Fixtures are ERB templates so we can use Ruby to build our test objects
- Objects primary keys (IDs) are missing
- Objects get a name reference (`joe` and `chris`)

Since the fixture files are processed as ERB templates, we can create dummy data with `Faker`, hash passwords, or even dynamically create way more records. `Faker` is a Ruby gem that can generate random data for the test suite. We can use it in fixtures once we add it to Gemfile:

```
# Gemfile

group :development, :test do
  gem "faker"
end
```

Here's how we would technically create 100 user records having a unique email address:

```
<% 1.upto(100) do |i| %>
user_<%= i %>:
  name: "User <%= i %>"
  email: <%= Faker::Internet.unique.email %>
<% end %>
```

Remember that the point of fixtures is to represent predictable data. I recommend creating only a small amount of most generic variations. Other specific objects can be made when needed by either modifying an existing fixture or even creating a completely new object with a standard Active Record:

```
class UserTest < ActiveSupport::TestCase
  setup do
    @user = users(:joe)
    @user.update(my_attribute: "New value")

    @other_user = User.create!(my_attribute: "Passed value")
  end

  test "#my_attribute" do
    assert_equal "New value", @user.my_attribute
    assert_equal "Passed value", @other_user.my_attribute
  end
end
```

So we shouldn't feel the need to always have a premade fixture that fits the test.

We also get a specific collection of fixtures by passing identifiers for all fixtures we want to work with:

```
class UserTest < ActiveSupport::TestCase
  test "find one" do
    assert_equal 2, users(:joe, :chris).length
  end
end
```

This helper is the most common way to reference fixtures, but there are other ways too. We can instruct Rails to create instantiated fixtures by setting `use_instantiated_fixtures` inside the `ActiveSupport::TestCase` test case:

```
class UserTest < ActiveSupport::TestCase
  self.use_instantiated_fixtures = true

  test "#confirmed?" do
    assert @joe.confirmed?
  end
end
```

```
end
end
```

Now each fixture is available as an instance variable and we can also access all of the fixtures and their attributes as hash named after the model:

```
class UserTest < ActiveSupport::TestCase
  self.use_instantiated_fixtures = true

  test "#name" do
    assert_equal "Joe", @users["joe"]["name"]
  end
end
```

If we only care about the fixtures hash, we can set `use_instantiated_fixtures` to `:no_instances`:

```
class UserTest < ActiveSupport::TestCase
  self.use_instantiated_fixtures = :no_instances

  test "#name" do
    assert_equal "Joe", @users["joe"]["name"]
  end
end
```

Instantiated fixtures come at a performance cost of traversing the database so they should only be used in very specific cases. I don't use them much, if at all. Instead, I do name my fixtures for the given test:

```
class UserTest < ActiveSupport::TestCase
  setup do
    @user = users(:peter)
    @admin = users(:joe)
  end

  test "#admin?" do
    assert_not @user.admin?
    assert @admin.admin?
  end
end
```

We might use the same fixture in different tests. In the test above the important thing about the user is if he's an admin. So I name a regular user `@user` and admin `@admin`. Sometimes, if the attribute is important we can also append it to the fixture name, like `joe_admin`. This kind of naming is the RSpec

equivalent of named subjects.

Finally, we can always reference fixtures indirectly without any helpers at all with plain Active Record queries:

```
test "find one" do
  assert_equal 10, User.all.count
end
```

They are just (unordered) records in the database after all. And this brings us back to their IDs and how can we use them in relationships.

5.1.1. Relationships

Since fixtures are modeling our little world, we'll also need to handle relationship associations like `belongs_to` and `has_many`. But to do that, we have to understand how fixtures get their IDs.

Rails fixtures come with stable automatically generated IDs. They are based on the type and the name of the fixture, so Rails easily knows what to put as a foreign key when they are referenced by name. This means a user fixture named `joe` will get an ID of `633107804` which is then used to fill in foreign keys.

Here's an example of a user belonging to a team via `TeamRole` model:

```
# app/models/user.rb
class User
  has_many :team_roles
  has_many :team_roles, dependent: :destroy
  has_many :teams, through: :team_roles
end

# app/models/team.rb
class Team
  has_many :team_roles, dependent: :destroy
  has_many :users, through: :team_roles
end

# app/models/team_role.rb
class TeamRole
  belongs_to :team
  belongs_to :user
end
```

And their corresponding fixture files:

```
# test/fixtures/users.yml
joe:
  name: Joe
  email: joe@company.com

# test/fixtures/teams.yml
big_corp:
  name: Big Corp.

# test/fixtures/team_roles.yml
big_corp_owner:
  team: big_corp
  user: joe
  role: :owner
```

Rails is clever enough to reference the user fixture named `joe` and since the ID is predictable, it doesn't even have to remember what fixture has what ID. We can also include the fixture class:

```
# test/fixtures/team_roles.yml
big_corp_owner:
  team: big_corp (Team)
  user: joe (User)
  role: :owner
```

We need to reference the class when using polymorphic relationships (where one model belongs to several other models).

We can also get the exact ID with `ActiveRecord::FixtureSet.identify`:

```
# test/fixtures/team_roles.yml
big_corp_owner:
  team_id: <%= ActiveRecord::FixtureSet.identify(:big_corp) %>
  user_id: <%= ActiveRecord::FixtureSet.identify(:joe) %>
  role: :owner
```

A non-standard identifier can be provided as the second argument:

```
ActiveRecord::FixtureSet.identify(:joe, :uuid)
```

Automatic fixture IDs are quite clever but you might still be wondering if you can provide IDs to fixtures yourself. Well, you can. If you add your ID and reference the relationship properly from the other fixture files, things will mostly work:

```
# test/fixtures/users.yml
joe:
  id: 1
  name: Joe
  email: joe@company.com

# test/fixtures/teams.yml
big_corp:
  id: 1
  name: Big Corp.

# test/fixtures/team_roles.yml
big_corp_owner:
  team_id: 1
  user_id: 1
  role: :owner
```

But let's try another example. What if we want to share the same set of multi-level topics between all environments?

Looks like this could work:

```
# test/fixtures/topics.yml
books:
  id: 1
  name: Books

fiction:
  id: 2
  name: Fiction
  parent_id: 1
```

But it doesn't. Referencing by ID in the same file won't work because we work with unordered fixtures. However, Rails does support ordered fixtures using OMAP YAML type:

```
# test/fixtures/topics.yml
--- !omap
- books:
  id: 1
  parent_id: NULL
  title: Books
- fiction:
  id: 2
  parent_id: 1
```

```
title: Fiction
```

If we switch to ordered fixtures we can now safely reference fixtures by IDs everywhere.

Still, we should prefer fixtures without IDs. There are two other good features they get. The first one lets us model `has_and_belongs_to_many` (HABTM) relationships as inline lists. Our `TeamRole` association could be completely skipped if we don't use IDs and don't need to provide other attributes:

```
# test/fixtures/posts.yml
first:
  title: A very first post
  tags: marketing, tech

# test/fixtures/tags.yml
marketing:
  name: Marketing

tech:
  name: Tech
```

The `tags` above simply list associated tags directly even though it's a HABTM relationship with an extra table.

The second feature is auto-filling timestamp columns like `created_at` and `updated_at` with `Time.now`. A small but neat feature.

5.1.2. Transactional fixtures

By default, test cases don't run in transactions. All of the fixtures are simply loaded at the start of each test and then deleted at the end. This can be changed with `use_transactional_tests` settings which will run the tests of the test case in transactions:

```
class UserTest < ActiveSupport::TestCase
  self.use_transactional_tests = true

  test "deleting users" do
    assert_not User.all.empty?
    User.destroy_all
    assert User.all.empty?
  end

  test "users still exist" do
    assert_not User.all.empty?
  end
end
```



```
end
```

The test case looks practically the same and we get the same result. So what exactly changed?

Instead of deleting all records and reinserting them, Rails will wrap the test with a transaction that will be rolled back at the end. This is quite a bit faster operation but not without its drawbacks.

Here's why using transactional fixtures is sometimes problematic:

- Nested transactions wouldn't commit until the parent transaction is committed which means the transaction under test wouldn't be committed before the teardown step. This means you cannot use this feature when testing transactions.
- Not all databases support transactions (like MySQL MyISAM)

If you preload your fixtures (like in a Rake task) you can set `self.pre_loaded_fixtures` to true and you'll get access to your fixtures even if they are already in the database.

I would treat transactional fixtures as an option for optimizing specific tests but chances are you won't need them.

This feature can now also be removed for a given database since Rails 8:

```
class UserTest < ActiveSupport::TestCase
  self.use_transactional_tests = true
  skip_transactional_tests_for_database :shared_database

  ...
end
```

5.2. Active Storage

Active Storage lets us quickly add file attachments to any Active Record model. This convenience means the storage is completely separated and has its own database tables. To add fixtures for uploaded files, we have to add fixtures for Active Storage attachments and blobs directly.

Active Storage fixtures live in `fixtures/active_storage` and come in two files, one for attachments and one for blobs. Attachments are the context records connected to our models while blobs are representing files themselves.

Let's say the user has an attached avatar:

```
# app/models/user.rb
class User < ApplicationRecord
  has_one_attached :avatar do |attachable|
```

```
    attachable.variant :thumb, resize_to_limit: [ 160, 160 ]
  end
end
```

The user model will come with a fixture file that won't mention the avatar at all:

```
# fixtures/users.yml
joe:
  name: Joe
  email: <%= Faker::Internet.unique.email %>

peter:
  name: Peter
  email: <%= Faker::Internet.unique.email %>
```

That's because the avatar will be referenced the other way around from the attachment:

```
# fixtures/active_storage/attachments.yml

joe_avatar_attachment:
  name: avatar
  record: joe (User)
  blob: joe_avatar_blob

# fixtures/active_storage/blobs.yml

joe_avatar_blob: <%= ActiveStorage::FixtureSet.blob filename: "joe.jpg" %>
```

A file from `test/fixtures/files` named `joe.jpg` will be used for the blob.

We could also provide file attributes directly:

```
joe_avatar_blob:
  key: avatar_john_key
  filename: avatar.jpg
  content_type: image/jpeg
  metadata:
    identified: true
    analyzed: true
  byte_size: 12345
  checksum: <%= Base64.encode64(Digest::MD5.digest('avatar-content')) %>
  created_at: <%= Time.now.to_s(:db) %>
```

But note that variants won't be part of fixtures. If you really need to test variants, you'll do so directly in tests.

5.3. Action Text

Similarly to Active Storage attachments, Action Text is too decoupled from the models themselves. So if we want to create an Action Text fixture, we do it inside `action_text/rich_texts.yml`.

As an example, let's say we have a `BlogPost` model with attached Action Text `content` field.

```
# app/models/blog_post.rb
class BlogPost < ApplicationRecord
  has_rich_text :content
end
```

The fixture file for the blog post would look like this:

```
# blog_posts.yml
published:
  title: First published post
  slug: first-post
  published_at: <%= DateTime.current %>

scheduled:
  title: Scheduled into the future
  slug: second-post
  published_at: <%= 1.day.from_now %>

draft:
  title: Just a draft
  slug: third-post
```

And the file for the attached `content` Action Text field is:

```
# action_text/rich_texts.yml

published_post_text:
  record: published (BlogPost)
  name: content
  body: <p>Rich HTML text</p>

scheduled_post_text:
  record: scheduled (BlogPost)
  name: content
```

```
body: <p>Rich HTML text</p>
```

```
draft_post_text:  
  record: draft (BlogPost)  
  name: content  
  body: <p>Rich HTML text</p>
```

In the example, `published_post_text` is just a reference name as with other fixtures. I always prefer good names to numbers like `one` and `two`.

5.4. Helpers

Since we might depend on Ruby to build our fixtures up, Rails allows us to define global helpers for fixtures. You can think of them as Action View helpers but for fixtures.

The `ActiveRecord::FixtureSet.context_class` module defines the context for fixtures so to make a custom helper, you'll need your helper module to join this context:

```
# test_helper.rb  
  
module FixtureFileHelpers  
  def password_digest_for(class, password)  
    Devise::Encryptor.digest(User, password)  
  end  
  
  def file_sha(path)  
    OpenSSL::Digest::SHA256.hexdigest(File.read(Rails.root.join('test/fixtures', path)))  
  end  
end  
  
ActiveRecord::FixtureSet.context_class.include FixtureFileHelpers
```

Then your helper methods become available to all fixtures at once:

```
# test/fixtures/users.yml  
<% password = "test123456" %>  
  
joe:  
  name: Joe  
  email: <%= Faker::Internet.unique.email %>  
  encrypted_password: <%= password_digest_for(User, password) %>  
  ...
```

5.4.1. Paths

To add more non-standard fixtures path, extend `fixture_paths`:

```
# test_helper.rb

ActiveSupport::TestCase.fixture_paths << "component1/test/fixtures"
ActiveSupport::TestCase.fixture_paths << "component2/test/fixtures"
```

NOTE Before Rails 7.1, this was a single path called `fixture_path`.

Chapter 6. Model tests

Model tests should test your models and business logic. This means Active Model, Active Record, or any other Ruby object usually found in `app/models`. These tests are straightforward to write and fast to execute, so they should be the backbone of your Rails application test suite. They are the simplest tests of all because you can treat them as plain Minitest tests.

We primarily test our business operations with occasional tests for scopes and validations when needed. Testing for persistence methods, finders, and most associations is usually not required but remember that just because an association has tests in Rails, it doesn't mean it protects your application design in any way.

Since I tend to write model tests in a similar order as the model itself let's start with scopes:

```
# test/models/car_test.rb
require "test_helper"

class CarTest < ActiveSupport::TestCase
  test ".available" do
    assert_equal 3, Car.available.count
  end

  test ".sold" do
    assert_equal 1, Car.sold.count
  end

  test ".luxury" do
    assert_equal 1, Car.luxury.count
  end

  ...
end
```

Fixtures are great for scopes because the alternative would be to create a lot of records on the spot. Let's see one more example, this time for blog posts:

```
# test/models/blog_post_test.rb
require "test_helper"

class BlogPostTest < ActiveSupport::TestCase
  test ".statuses" do
    assert_equal 3, BlogPost.statuses.count
  end

  test ".status" do
```

```

    assert_equal 1, BlogPost.status(:draft).count
    assert_equal 2, BlogPost.status(:published).count
  end

  test ".draft" do
    assert_equal 1, BlogPost.draft.count
  end

  ...
end

```

After scopes and other class methods, we might want to test important validations or validations with custom rules.

```

require "test_helper"

class BlogPostTest < ActiveSupport::TestCase
  setup do
    @blog_post = blog_posts(:first_published)
  end

  ...

  test "validation should succeed for valid slug" do
    post = BlogPost.new(title: "Post", slug: "post")
    assert post.valid?
  end

  test "slug should be auto-generated" do
    post = BlogPost.new(title: "Post", slug: nil)
    post.save!
    assert post.reload.slug
  end
end

```

If we would use `validate` with a custom validation, we might already have a specific test for the validator class. Still, this doesn't test our model behavior and similarly, that's why a test of validation in Rails doesn't test our model behavior. We should at the very least have a basic test for `validates_format_of` since it accepts a regular expression.

As I mentioned in the beginning it's not necessary to test for having associations. And by implementing fixtures we know our relations work. But if we want to test it it could look something like this:

```

# test/models/post_test.rb
require 'test_helper'

```

```

class PostTest < ActiveSupport::TestCase
  fixtures :posts, :comments

  test "should belong to user" do
    post = posts(:post_one)
    assert_equal users(:john), post.user
  end

  test "should have many comments" do
    post = posts(:post_one)
    assert_equal 1, post.comments.count
    assert_includes post.comments, comments(:comment_one)
  end
end

```

The rest of the test cases would be your custom methods. I usually use the Ruby notation for public and private methods as the description of the tests:

```

require "test_helper"

class BlogPostTest < ActiveSupport::TestCase
  setup do
    @blog_post = blog_posts(:first_published)
  end

  test ".all" do
    assert_equal 3, BlogPost.all.count
  end

  test "#status" do
    @blog_post.published_at = nil
    assert_equal :draft, @blog_post.status
    @blog_post.published_at = 1.day.ago
    assert_equal :published, @blog_post.status
    @blog_post.published_at = DateTime.current
    assert_equal :published, @blog_post.status
    @blog_post.published_at = 1.day.from_now
    assert_equal :scheduled, @blog_post.status
  end

  test "#published?" do
    @blog_post.published_at = DateTime.current
    assert @blog_post.published?
    @blog_post.published_at = 1.day.from_now
    assert_not @blog_post.published?
  end
end

```



```
end
end
```

Don't be afraid to bundle more assertions together. This makes the test suite faster.

6.1. Callbacks

Models in Rails often come with callbacks. We don't usually test callbacks in isolation, but since fixtures insert database records without running the callbacks, we might need a way to run them. In Rails, we can run `run_callbacks` as a block to rerun a callback.

Here's one example of testing the effects of the create callback:

```
class UserTest < ActiveSupport::TestCase
  test "should re-run after_create callback" do
    user = users(:john)

    # Re-run the after_create callback
    user.run_callbacks(:create) do
      # run before create first
      ...
      # run after create last
    end

    # Test the effects of the callback
    assert user.welcome_email_sent?
  end
end
```

This way we can save on creating an extra record just to test the callback effect, but it's also acceptable to create a new record with `create!` to make sure everything happens as expected. Providing a block with extra code is optional.

6.2. Concerns

We can test model concerns either directly in isolation or indirectly via models we use them in. If we want to test them in isolation, we can extend a random `Object` with our concern module like this:

```
# app/models/concerns/greetable.rb
module Greetable
  def introduce_yourself
    "Hello, I am #{name}"
  end
end
```

```
# test/models/concerns/introduction_test.rb
require "test_helper"

module DependentMethods
  def name
    "Joe"
  end
end

class IntroductionTest < Minitest::Test
  def setup
    @greetable = Object.new
    @greetable.extend DependentMethods
    @greetable.extend Greetable
  end

  test "will be able to introduce himself"
    assert_equal "Hello, I am Joe", @greetable.introduce_yourself
  end
end
```

Still, a lot of times it's better to test it as a behavior on the actual model we care about;

```
# app/models/user.rb
class User
  include Introduction
end

# test/models/user.rb
class UserTest < Minitest::Test
  def setup
    @user = User.new(name: "Joe")
  end

  test "will be able to introduce himself"
    assert_equal "Hello, I am Joe", @user.introduce_yourself
  end
end
```

We should care more about the functionality of the `User` model rather than what's happening behind the scenes. But if the concern is more complex it makes sense to have a separate test.

NOTE

Minitest doesn't come with shared examples, you would need to repeat the tests for every occasion.

6.3. Attachments

Active Storage makes it dead simple to start with file uploads in Rails. Let's say our users come with avatars that should also have smaller thumbnails:

```
class User < ApplicationRecord
  has_one_attached :avatar do |attachable|
    attachable.variant :thumb, resize_to_limit: [ 160, 160 ]
  end
end
```

We can then test if an avatar is attached and also some of the variant metadata:

```
require "test_helper"

class User::AvatarTest < ActiveSupport::TestCase
  include ActionDispatch::TestProcess

  setup do
    @user = users(:john)
    @user.avatar.attach(io: File.open(Rails.root.join("test", "fixtures", "files",
"avatar.jpg")), filename: "avatar.jpg", content_type: "image/jpeg")
    @user.reload
  end

  test "user has correct avatar" do
    assert @user.avatar.attached?
    assert_equal "avatar.jpg", @user.avatar.filename.to_s
  end

  test "avatar thumbnail variant is 160x160" do
    thumbnail = @user.avatar_thumbnail
    assert_equal 160, thumbnail.metadata[:width]
    assert_equal 160, thumbnail.metadata[:height]
  end
end
```

6.4. Queues and Jobs

Testing jobs is a two-sided thing. On one hand, we test that a particular job has been scheduled and on the other, we still have to test the implementation of a said job.

The default delivery will depend on your queue adapter:

```
# config/environments/test.rb

Rails.application.configure do
  # Possibly :inline
  config.active_job.queue_adapter = :solid_queue
  ...
end
```

Using an inline adapter will always run your jobs in the moment so you can test what you need. The alternative is to test a job is enqueued in your business code and check for the job's action in a specific job test.

To do that Rails gives us a specific assertion called `assert_enqueued_jobs` which will check the number and possibly type of scheduled jobs. We use it as a block:

```
require "test_helper"

class UserTest < ActiveSupport::TestCase
  include ActiveJob::TestHelper

  test "creating a message enqueues to push later" do
    assert_enqueued_jobs 1, only: [ User::SendWelcomeEmailJob ] do
      User.create!(name: "Joe")
    end
  end
end
```

Testing the job itself can be tested as a regular Ruby object by calling `perform_now` on the job instead of `perform_async`:

```
# test/jobs/user/send_welcome_email_job_test.rb

require "test_helper"

class User::SendWelcomeEmailJobTest < ActiveSupport::TestCase
  test "creating a message enqueues to push later" do
    user = users(:joe)
    User::SendWelcomeEmailJob.perform_now(user)
    assert user.reload.welcome_email_sent?
  end
end
```

This way we can test anything even without an inline adapter which might run jobs that don't need to be run during tests making the test suite faster.

6.5. Action Cable

We can test Action Cable connections for being connected, subscribed to channels, and receiving broadcast messages. Let's say we are implementing a chat application with a `ChatChannel` that users can subscribe to. Once connected, they will receive a notification about the messages in the room.

Since Action Cable connection will be tied to a `current_user`, let's assume we can find it based on a session cookie:

```
module ApplicationCable
  class Connection < ActionCable::Connection::Base
    identified_by :current_user

    def connect
      self.current_user = find_user
    end

    private

    def find_user
      if session = find_session_by_cookie
        session.user
      else
        reject_unauthorized_connection
      end
    end

    def find_session_by_cookie
      if token = cookies.signed[:session_token]
        Session.find_by(token: token)
      end
    end
  end
end
```

The channel implementation will expect a chat room to subscribe to:

```
# app/channels/chat_channel.rb
class ChatChannel < ApplicationCable::Channel
  def subscribed
    if @room = find_room
      stream_for @room
      # or
      stream_from "chat_#{params[:room_id]}"
    else

```

```

    reject
  end
end

private

def find_room
  Room.find_by(id: params[:room_id])
end
end

```

Once we have our implementation we can start writing tests subclassing `ActionCable::Channel::TestCase`:

```

require "test_helper"

class ChatChannelTest < ActionCable::Channel::TestCase
  setup do
    stub_connection(current_user: users(:joe))
    @room = rooms(:introductions)
  end

  test "user can join a chat room" do
    subscribe room_id: @room.id

    assert subscription.confirmed?
    assert_has_stream_for @room
    # Or
    assert_has_stream "chat_1"
  end

  test "user cannot join a non existing room" do
    subscribe room_id: -1

    assert subscription.rejected?
    assert_no_stream
  end
end

```

The `stub_connection` helper creates an Action Cable connection and `subscribe` subscribes to the `ChatChannel`.

To test the subscription is successful, we can assert `subscription.confirmed` or `subscription.rejected`. To test the subscription subscribes to a specific stream, we can use `assert_has_stream_for`, `assert_has_stream` or `assert_no_stream`.

Next, we need to test broadcasting:

```
require "test_helper"

class ChatTest < ActionCable::Channel::TestCase
  test "one chat message received" do
    # code triggering the broadcast, perhaps a job
    # e.g. ActionCable.server.broadcast "room_1", message: "Hello"
    assert_broadcasts ChatChannel.broadcasting_for("room_1"), 1
  end

  test "broadcasts a chat message" do
    # code triggering the broadcast
    assert_broadcast_on ChatChannel.broadcasting_for("room_1"), message: "Hello
everyone!"
  end

  test "no chat message published" do
    assert_no_broadcasts ChatChannel.broadcasting_for("room_1")
  end
end
```

We can use `assert_broadcasts` with the number of received messages, `assert_broadcast_on` for an exact message or `assert_no_broadcasts`. We can also use them with a passed block.

6.6. Emails

The default delivery method for Action Mailer in the test environment is `:test` which won't attempt to send the email, but will populate `ActionMailer::Base.deliveries` array which we can check for successful deliveries:

```
# config/environments/test.rb

Rails.application.configure do
  # Tell Action Mailer not to deliver emails to the real world.
  # The :test delivery method accumulates sent emails in the
  # ActionMailer::Base.deliveries array.
  config.action_mailer.delivery_method = :test
  ...
end
```

Rails also gives us a specific assertion called `assert_emails` which will check the number of scheduled deliveries. We use it as a block:

```
# test/jobs/user/send_welcome_email_job_test.rb
```

```
require "test_helper"

class User::SendWelcomeEmailJobTest < ActiveSupport::TestCase
  test "send a welcome email" do
    user = users(:joe)

    assert_emails 1 do
      User::SendWelcomeEmailJob.perform_now(user)
    end
  end
end
```

If we want to test and check the mailer details, we can ask the mailer:

```
require "test_helper"

class NotificationMailerTest < ActionMailer::TestCase
  def setup
    @user = users(:one)
  end

  test "#welcome" do
    email = NotificationMailer.welcome(@user)

    assert_equal email.to, [@user.email]
    assert_equal email.from, ["info@testdrivingrails.com"]
    assert_equal email.subject, "Welcome"
    assert_match "Thanks you", email.body.encoded
  end
end
```

To test time-sensitive tokens in emails, use `freeze_time` helper so the token generation inside the test and inside the job uses the same time.

To configure the hostname for links provided in emails, set the `default_url_options` for Action Mailer:

```
# config/environments/test.rb
Rails.application.configure do
  config.action_mailer.default_url_options = { host: "www.example.com" }
end
```


6.7. Services

If your application comes with other business objects and operations elsewhere in the code, like with services in `app/services` or form objects in `app/forms`, then you should create the corresponding `test/services` and `test/forms` directories.

Tests themselves would usually resemble model tests and you would depend on the same assertions, test doubles, and fixtures. Since we have seen that Minitest runs automatically every test case that's already loaded, we only need to require the custom paths in `test_helper.rb`:

```
# test/test_helper.rb
Dir[Rails.root.join("test/services/**/*.rb")].each { |f| require f }
```

6.8. Time

Some tests are time-sensitive. Luckily for us, Rails has nice time helpers like `freeze_time`, `travel_to`, `travel_back`, and `travel`:

```
Time.current      # => Sat, 09 Nov 2013 15:34:49 EST -05:00
travel_to Time.zone.local(2004, 11, 24, 01, 04, 44)
Time.current      # => Wed, 24 Nov 2004 01:04:44 EST -05:00
Date.current      # => Wed, 24 Nov 2004
DateTime.current  # => Wed, 24 Nov 2004 01:04:44 -0500
```

They also come in a block variant isolating the action we need:

```
# Travel forward in time
Time.current # => Sat, 09 Nov 2013 15:34:49 EST -05:00
travel 1.day do
  User.create(created_at: Time.current) # => Sun, 10 Nov 2013 15:34:49 EST -05:00
end
Time.current # => Sat, 09 Nov 2013 15:34:49 EST -05:00
```

6.9. HTTP

Testing external HTTP interactions is likely the most dreaded part of web application testing. Running real HTTP requests is slow, not deterministic, prone to errors, and requires an internet connection in the first place. Luckily there are some good options for avoiding all of that to ensure our tests are fast and deterministic. The ones you might want to look at are WebMock and VCR.

WebMock* is exactly what it sounds like, a mocking library tailored to web requests. In principle, it's not too different from mocking we have already seen, but lets us stub generic requests by URL.

WebMock doesn't require an internet connection but requires us to model the HTTP responses we want to use in tests.

```
# Gemfile
gem "webmock", only: :test
```

The common use case for WebMock is to stub a particular URL and respond with a hand-crafted response:

```
class PriceFinderTest < ActionMailer::TestCase
  test "finds the price of the car on the page" do
    WebMock
      .stub_request(:get, "https://cars.com/vehicle/tesla/234")
      .to_return(
        status: 200,
        body: "<body><h1>Tesla 3</h1><p>Price: $39000</p><body>",
        headers: { content_type: "text/html" }
      )

    price_usd = PriceFinder.new("https://cars.com/vehicle/tesla/234")
    assert_equal price_usd, 39_000
  end
end
```

WebMock's `stub_request` will stub any outgoing request based on the HTTP method and URL. Then it returns a fake response that the code under test will work with. This is possible since the library stubs the `Net::HTTP` client and other HTTP gems under the hood automatically. Still, it's a good idea to check that your HTTP client of choice is supported.

Let's see one more example, this time by posting some data:

```
class CarBidTest < ActionMailer::TestCase
  test "makes a bid in auction" do
    WebMock
      .stub_request(:post, "https://api.cars.com/bids/234")
      .with(
        body: { offer_usd: 50_000 }.to_json,
        headers: {
          'Content-Type' => "application/json",
          'Authorization' => "Bearer mysecrettoken"
        }
      )
      .to_return(
        status: 200,

```

```

      body: { result: :ok }.to_json,
      headers: { content_type: "application/json" }
    )

    bid = CarBid.new(car_id: 234, offer_user: 50_000)
    assert bid.success?
  end
end

```

The request from `CarBid` won't make it to the real site, but `stub_request` together with `with` lets us check the right data are posted, including the correct authentication headers.

So while we'll lose some accuracy of what's returned, WebMock tests are easy to maintain and work with, especially in the long term. We can also configure tests to never attempt to do real requests:

```

# test/test_helper.rb

...
# Optional: Disabling real network connections for tests
WebMock.disable_net_connect!(allow_localhost: true)

```

NOTE

WebMock is the library of choice in some latest products from 37signals, a company behind the Rails framework.

VRC records your test suite's HTTP calls and replay them later during future test runs. It's an alternative approach to HTTP mocking that gets you more accurate and still deterministic responses. While still pretty easy to start, VRC tend to be more difficult to maintain as the time goes.

Here's a small configuration for using VCR:

```

# Gemfile
gem "vcr", only: :test

# test/test_helper.rb
require "vcr"

VCR.configure do |config|
  config.cassette_library_dir = "fixtures/vcr_cassettes"
  config.hook_into :webmock
  config.ignore_localhost = true
end

```

As you can see VRC can use WebMock under the hood to provide the prerecorded interactions called cassettes. The `cassette_library_dir` config holds the path to the cassettes fixtures which needs to be

checked in to source control like git. The `hook_into` part pairs VRC with WebMock, but more HTTP stubbing libraries are supported.

The test itself simply specifies which cassette to use and wraps any logic with HTTP calls:

```
# test/models/iana_test.rb

class IanaTest < Test::Unit::TestCase
  def test_reserve_domains
    VCR.use_cassette("iana/reserved") do
      response = Net::HTTP.get_response(URI("http://www.iana.org/domains/reserved"))
      assert_match /Example domains/, response.body
    end
  end
end
```

The first run of the test will fetch an actual page from IANA website while subsequent runs will fetch the cached version from the `cassette_library_dir` directory which is by default `test/cassettes`.

Here's a full example of a noname service with a POST request:

```
# app/services/example_service.rb
require "net/http"

class ExampleService
  def self.post_request(data)
    uri = URI("https://jsonplaceholder.typicode.com/posts")
    http = Net::HTTP.new(uri.host, uri.port)
    http.use_ssl = true
    request = Net::HTTP::Post.new(uri.path, { "Content-Type" => "application/json" })
    request.body = data.to_json
    response = http.request(request)
    JSON.parse(response.body)
  end
end
```

And the associated VRC test:

```
# test/services/example_service_test.rb
require "test_helper"
require "example_service"

class ExampleServiceTest < Minitest::Test
  def setup
    @data = { title: "foo", body: "bar" }
```

```
end

def "post request"
  VCR.use_cassette("example_service/post_request") do
    response = ExampleService.post_request(@data)
    assert_equal "foo", response["title"]
    assert_equal "bar", response["body"]
  end
end
end
```

When your code changes, you'll need to rerecord the related cassettes. Simply delete the cassette file in question and rerun your test suite.

VCR works great in simple cases, but more complex examples can get harder to maintain. For example, testing a payment processing integration might require a special test setup on the provider side which could be lost over time.

TIP	Always document how to prepare the test environment to get the mocked responses. You'll save your colleagues a lot of time.
------------	---

Personally I used both approaches and I wouldn't be dogmatic about either of them. You can even mix and match both in one test suite if that makes sense.

Chapter 7. Controller tests

Rails applications are first and foremost web applications handling client requests. Every corner of your application goes through the controller layer which:

- accepts requests from clients,
- authenticate and authorize the client,
- redirects,
- handles form inputs,
- calls business logic,
- assign variables for the view layer,
- and renders a response

We can test all of these with integration and controller tests. Technically speaking they are both based on `ActionDispatch::IntegrationTest` but Rails splits them into two different kinds of tests:

- **Integration tests** test controller flow through an application. This could be a user adding items to a card and doing a checkout at the end of a user going through a multi-step registration.
- **Functional tests**, also called just controller tests, are integration tests for a single controller. Instead of testing more complex flows, they focus on testing different controller actions and their conditions.

Since these tests can do the same thing a lot of Rails applications out there have only one set of these tests. The difference is only in their intent.

7.1. Actions

Controller tests usually start with a visit to a named route and are followed by assertions about the page. Let's see what this might look like.

This is a small blog post controller you could find in the wild:

```
# app/controllers/blog_posts_controller.rb
class BlogPostsController < ApplicationController
  layout "blog"
  before_action :set_blog_post, only: %i[ show ]

  # GET /blog
  def index
    @query = BlogPost.published.order(id: :desc).with_rich_text_content
    @pagy, @blog_posts = pagy(@query)
  end
end
```

```

# GET /blog/:slug
def show
  @blog_post.increment!(:hits)
end

private
def set_blog_post
  @blog_post = BlogPost.published.find_by!(slug: params[:slug])
end
end

```

And the associated test:

```

# test/controllers/blog_posts_controller_test.rb
require "test_helper"

class BlogPostsControllerTest < ActionDispatch::IntegrationTest
  setup do
    @published_post = blog_posts(:first_published)
    @draft = blog_posts(:draft)
    @scheduled_post = blog_posts(:scheduled)
  end

  test "should get index" do
    get blog_posts_url
    assert_response :success
    assert response.body.include?(@published_post.title)
    assert_not response.body.include?(@draft.title)
    assert_not response.body.include?(@scheduled_post.title)
  end

  test "should show blog post" do
    get blog_post_url(@published_post)
    assert_response :success
    assert_equal 1, @published_post.reload.hits
  end
end

```

To make testing controllers simple Rails comes with handy test helpers that can initiate REST requests by accepting a standard Rails route path:

- `get` for GET requests like `index` and `show` actions
- `post` for POST requests like `create` action
- `put` for PUT requests like `update` action

- `patch` for PATCH requests like `update` action
- `delete` for DELETE requests like `destroy` action
- `head` for HEAD requests

The helpers also accepts a few options.

We get an `xhr` flag for Ajax requests and `as` for content types:

```
test "ajax request" do
  @published_post = blog_posts(:first_published)
  get blog_post_url(@published_post), xhr: true

  assert_equal "A first post", response.body
  assert_equal "text/javascript", response.media_type
end

test "json request"
  assert_difference("BlogPost.count") do
    post admin_blog_posts_url, params: { blog_post: { title: "Ahoy!" } }, as: :json
  end
end
```

We get `params` to provide submitted form data for `post`, `put` and `patch`:

```
test "should create unpublished blog post" do
  log_in(@admin)
  assert_difference("BlogPost.count") do
    post admin_blog_posts_url, params: { blog_post: { slug: @blog_post.slug, title:
    @blog_post.title } }
  end

  assert_redirected_to admin_blog_post_url(BlogPost.last)
  assert_nil BlogPost.last.published_at
end
```

And also `headers` for providing any required request headers:

```
test "ajax request" do
  @published_post = blog_posts(:first_published)
  get get blog_post_url(@published_post), headers: { "HTTP_REFERER" =>
  "http://example.com/home" }
  assert_equal "A first post", response.body
end
```


You might have noticed that I am using URL helpers (`blog_post_url`) instead of path helpers (`blog_post_path`). But wouldn't path helpers work just the same?

```
# Relative paths
/users/1/edit

# Absolute URLs
http://localhost:3000/users/1/edit
```

Yes, they would. But since paths are relative whereas URLs are absolute, URLs offer just a bit more of a real environment to help us with test cases where protocol or host is relevant.

However, it's important to check we have the host set up correctly, otherwise we might run issues with cookies not being persisted across requests.

Here's how we can set the right URL options for tests:

```
# config/environments/test.rb
Rails.application.configure do
  routes.default_url_options = { host: "www.example.com" }
end
```

7.2. Responses

Once a controller action is executed, its HTTP request is available as `request`, response as `response`, and controller as `controller`. We'll also get `flash`, `cookies`, and `session` hashes populated with the respective data.

To assert what happened we get the `assert_response` assertion which takes either an exact status code or its symbolic representation:

- `:success` - 2xx status codes like 200 OK.
- `:redirect` - 3xx status codes like 302 Found.
- `:missing` - 404 status code
- `:error` - 5xx status codes
- `:unauthorized` - 401 status code
- `:forbidden` - 403 status code
- `:created` - 201 status code
- `:no_content` - 204 status code

If there is a redirect, we can assert the redirection with `assert_redirected_to` and run the

`follow_redirect!` helper in case we want to continue with the follow-up request to assert the final response code:

```
# test/controllers/admin/blog_posts_controller_test.rb
require "test_helper"

class Admin::BlogPostsControllerTest < ActionDispatch::IntegrationTest
  test "should create post" do
    assert_difference("BlogPost.count") do
      post admin_blog_posts_url, params: { blog_post: { slug: "hello-world", title:
"Hello world!" } }
    end

    # assert redirect
    assert_redirected_to :admin_blog_posts_path

    # continue redirect
    follow_redirect!
    assert response.body.include?("Hello world")
    assert_includes response.body, "Hello world"
  end
end
```

We usually check for a status code and when it makes sense also the return document body with `assert_includes` and `refute_includes`. There is even a helper to check for assigned instance variables in controllers:

```
test "should list posts" do
  get blog_posts_url
  assert_equal 5, assigns(:posts).count
end
```

Using `assigns` might feel like we are testing more of an implementation again, but all of these techniques are tools in a toolbox.

TIP

Integration tests are quite fast so a good opportunity to add some extra assertions. Don't worry to add some Active Record assertions if they make sense to you.

If a response came with a flash message, we can assert it as any other text or use the prepopulated `flash` hash:

```
# test/controllers/admin/blog_posts_controller_test.rb
require "test_helper"
```

```

class Admin::BlogPostsControllerTest < ActionDispatch::IntegrationTest
  test "should create post" do
    assert_difference("BlogPost.count") do
      post admin_blog_posts_url, params: { blog_post: { slug: "hello-world", title:
"Hello world!" } }
    end

    assert_redirected_to admin_blog_posts_path
    assert_equal "Post was successfully created.", flash[:notice]
  end
end

```

As you can see, flashes are simple. But what about cookies?

7.3. Cookies

Cookies are small pieces of data stored on the client's browser often used to implement sessions or user preferences. Rails has several types of cookies, like signed and permanent cookies, but it only supports overriding regular cookies in tests.

Let's imagine a scenario where we test the Rails 8 standard auth generator:

```

class SessionsControllerTest < ActionDispatch::IntegrationTest
  test "should create a new session" do
    user = users(:admin)
    post session_url, params: { session: { email: user.email, password: "123123" } }
    follow_redirect!
    assert_response :success
    assert_equal "/dashboard", path
  end

  test "should not sign in user with wrong password" do
    post session_url, params: { session: { email: user.email, password: "123" } }
    assert_response :unprocessable_entity
    assert_select "div.alert", "Invalid Email or password."
  end
end

```

As you can see no direct cookie management in sight, even though the session is created by assigning a signed permanent session:

```

# somewhere in Authentication module
cookies.signed.permanent[:session_id] = { value: session.id, httponly: true, same_site:
:lax }

```

The cookie management in tests is handled by Rails. If we continue making more requests the cookie will be preserved as long as we use path helpers or URL helpers with the same host (as mentioned previously):

```
class SessionsControllerTest < ActionDispatch::IntegrationTest
  test "should create session" do
    user = users(:admin)
    post session_url, params: { session: { email: user.email, password: "123123" } }
    follow_redirect!
    assert_response :success

    # still works
    get admin_dashboard_url
    assert_response :success
  end
end
```

However, we cannot directly access `cookies.signed` in tests. We can use `cookies[:name]` to either read the cookie from a previous request or use the same object to set it for the next request, but it won't set a signed cookie.

If you are using Devise, the biggest difference is that a Session object is not created in the database. Devise also provides helpers for a quick sign in and sign out so we should just use these for routes other than signing in itself:

```
# test/test_helper.rb
class ActiveSupport::TestCase
  ...

  # Devise helpers
  include Warden::Test::Helpers
  Warden.test_mode!

  teardown do
    Warden.test_reset!
  end

  def log_in(user, opts = {})
    login_as(user, opts)
  end

  def log_out(*scopes)
    logout(*scopes)
  end
end
```

Warden is the auth library used by Devise under the hood, so we are including its helpers and setting it to test mode.

7.4. Pagination

Since we used Pagy in one of the examples you might wonder what about testing pagination? If we list 10 blog posts do we need 11 fixtures to hit the pagination? I don't think we do. We can change the number of paginated items in tests:

```
# in an initializer
if Rails.env.test?
  Pagy::DEFAULT[:items] = 2
end

# in the test case
test "should get index and paginate results" do
  get blog_posts_url(page: 1)
  assert_includes response.body, "First post"
  assert_includes response.body, "Next page"

  # Request the second page
  get blog_posts_url(page: 2)
  assert_includes response.body, "Last post"
  refute_includes response.body, "Next page"
end
```

We can also test for assigned values:

```
test "should get index and paginate results" do
  # Request the second page
  get blog_posts_url(page: 2)

  # Check that pagy is assigned
  assert_not_nil assigns(:pagy)
  assert assigns(:pagy).is_a?(Pagy)

  # Check that the correct number of blog posts is assigned
  assert_not_nil assigns(:blog_posts)
end
```

Tests are often about one's preference and feeling of what feels right. As long as the test does a good job of preventing mistakes it's all that matters.

7.5. Attachments

Rails comes with a handy `fixture_file_upload` that can upload fixture files from the `test/fixtures/files` directory and create an instance of `ActionDispatch::Http::UploadedFile` for us:

```
# test/controllers/users_controller_test.rb
require "test_helper"

class UsersControllerTest < ActionDispatch::IntegrationTest
  setup do
    @user = users(:joe)
  end

  test "should upload avatar" do
    # takes file from test/fixtures/files/avatar.jpg
    file = fixture_file_upload("avatar.jpg", "image/jpeg")

    patch user_url(@user), params: { user: { avatar: file } }

    assert_redirected_to user_url(@user)
    @user.reload
    assert @user.avatar.exists?
  end
end
```

`ActionDispatch::Http::UploadedFile` comes with `original_filename`, `content_type` and `tempfile` params that we would otherwise have to provide ourselves.

7.6. Translations

Another question you might have is about asserting and working with translated strings. In my experience, people tend to write tests using `I18n` calls to assert whatever translated text is the current one. If this is the case, we can include `AbstractController::Translation` module to use the shorter `t` helper:

```
# test/test_helper.rb
class ActiveSupport::TestCase
  # Call I18n.t() with just t()
  include AbstractController::Translation

  ...
end
```

The same approach can also be taken in system tests which we'll look at next.

Chapter 8. System tests

System tests, which are usually known as end-to-end tests outside of Rails, are designed to test the entire application from the user's perspective. These tests simulate real user interactions by running a real or headless web browser to interact with the application, similarly as a user would. System tests cover all aspects of the application, including routing, controllers, views, models, and JavaScript. They ensure that the entire system works together seamlessly to provide the correct functionality.

Testing applications using system tests is extremely valuable but also costly. Browsers are brittle and slow. Test libraries also often don't support the latest versions you might use yourself on your computer. They are difficult to run in parallel. Any test that doesn't have to be an end-to-end test likely shouldn't be one. We shouldn't strive to catch issues like a broken layout and every possible failure scenario.

System tests are sanity checks. Can the user still register and finish onboarding? Can he assign some tasks to a project and mark them later as completed? Could he set some configuration options? Since browser tests are a bit more complex than anything we went through so far, let's talk a bit more about how things come together first.

8.1. Configuration

System tests in Rails are based on the Capybara test library and run with Selenium web drivers. Let's explain first these few moving parts that need to be understood when it comes to system testing.

- Selenium is a suite of tools for automating web browsers that allows you to interact with web applications the same way as your users. Apart from testing applications, Selenium is can also be used for other things, like web scraping.
- Capybara is a library to simplify the process of writing automated web-based tests in Ruby. It comes with a behavioral DLS that we can use to describe the interaction that will be run with a Selenium web driver.
- Web drivers are bridging the automated test suite with web browsers. They interpret the test commands and translate them into actions in the browser (like clicking a button or entering text).

The choice of a driver is important as they might not all support the same interactions. Most people go for ChromeDriver which integrates with Google Chrome, but there is also GeckoDriver for Mozilla Firefox and SafariDriver for Safari.

The test driver and screen size are some of the basic configuration options for Rails system tests. Since system testing involves browsers, it comes with a specific configuration at `test/application_system_test_case.rb`:

```
# test/application_system_test_case.rb

# Use only a single process for a browser test
```

```

ENV["PARALLEL_WORKERS"] ||= "1"
require "test_helper"

class ApplicationSystemTestCase < ActionDispatch::SystemTestCase
  DRIVER = if ENV["DRIVER"]
    ENV["DRIVER"].to_sym
  else
    :headless_chrome
  end

  driven_by :selenium, using: DRIVER, screen_size: [ 1200, 700 ] do |option|
    option.add_argument "--disable-search-engine-choice-screen"
    option.binary = ENV["CHROME_BIN"] if ENV["CHROME_BIN"]
  end

  def setup
    # Capybara.app_host = "http://localhost:3000"
  end

  def teardown
    # Capybara.reset_sessions!
    # Capybara.use_default_driver
  end
end

# Override in individual tests with Capybara.using_wait_time(20) {}
Capybara.default_max_wait_time = 15
Capybara.save_path = Rails.root.join("tmp/capybara")
Selenium::WebDriver::Chrome::Service.driver_path = ENV["DRIVER_PATH"] if ENV
["DRIVER_PATH"].present?

```

The most important bit is to always match the driver with the supported version of the browser. With Chrome auto-updates we can quickly get a newer version of Chrome locally without even realizing it. Then we might need to install an updated driver as well. For example, We can do that on macOS with **brew**:

```

$ brew update
$ brew upgrade chromedriver

```

You could also notice I also included an explicit option to choose the Chrome binary (passed as an option) and driver (set at the very bottom). This can solve issues when more versions of Chrome are installed and we'll take advantage of it when setting up Continuous Integration.

TIP

If Rails system tests are not enough, there are pure JavaScript testing frameworks for browser testing that might lead to better results. But also, some things are better tested in

person with a naked eye.

8.2. Actions

System tests are composed of actions, finders, and matchers. Since any system test needs to start on a particular page, let's look at actions first.

The `visit` action accepts a path to visit and follows to the specific location.

```
require "application_system_test_case"

class FiltersTest < ApplicationSystemTestCase
  test "seeing listings by availability" do
    visit bookings_path
    ...
    assert_current_path /sort=availability/
  end
end
```

As you can see there is also the related `assert_current_path` assertion to check for the page we are on by URL.

When we land on a page we might want to continue to a different page, fill in a form, or click on a button. To move using links on the page, we can use `click_on` which clicks both links and buttons, or specifically `click_link`:

```
click_link "Next"
click_button "Send"
```

Once we arrive we might need to fill in a form. That's what the `fill_in` action is for. The same action is also available on the input itself:

```
# name, id, or label text matching 'Name'
fill_in 'Name', with: 'Bob'

# will fill in `element` if it's a fillable field
element.fill_in with: 'Tom'
```

The element above is referring to the input element. We'll see how we can query and get elements from the page in the next section.

We also have the `choose` action for radio buttons:

```
# name, id, or label text matching 'Male'
choose('Male')

element.choose()
```

The `check` and `unchecked` actions for checkboxes:

```
# a name, id, or label text matching 'German'
check('German')

# will check `element` if it's a checkbox element
element.check()
```

And the `select` and `unselect` actions for selects:

```
# The option can be found by its text.
select 'March', from: 'Month'
```

In more complex forms we are also able to submit a file

```
# a name, id, or label_text matching 'My File'
attach_file("Example file", "/path/to/file.png")

element.attach_file("/path/to/file.png")

attach_file("/path/to/file.png") do
  find('#upload_button').click
end
```

To submit a form we can reuse the `click_on` action or use a specific `click_button` to submit a button:

```
click_on "Submit"
click_button "Save"
```

We could also submit the form using `send_keys`:

```
require "application_system_test_case"

class MessageTest < ApplicationSystemTestCase
  ...
end
```

```
test "submitting a message using cmd + Enter keys" do
  visit chat_path(id: @chat.id)
  fill_in "message", with: "New message"
  find("#message").send_keys([:command, :enter])

  assert page.has_selector?("p", text: "New message")
end
end
```

Apart from regular user actions, we can also use Capybara to work with scripts as follows:

```
# Use execute_script to click the button using JavaScript
execute_script("document.getElementById('my_button').click()")

# Use evaluate_script to get the text content of the paragraph
text = evaluate_script("document.getElementById('my_paragraph').innerText")
```

The `execute_script` action simply executes the passed JavaScript and `evaluate_script` will return the value.

Finally, we might want to jump to a specific part of the page to make the relevant elements visible with `scroll_to`:

```
# Scroll to the target element
scroll_to("#target")
```

8.3. Finders

To be able to act on-page elements, either with actions or assertions, we have to be able to find them. There are several Capybara finders we have at our disposal:

- `find`
- `find_field`
- `find_link`
- `find_button`
- `find_by_id`
- `first`
- `all`

They are all available on our implicit page object and can find elements based on text, CSS, or XPath:

```
find("By text")
find(:css, "#by_css")
find(:xpath, "//*[@id='xpath']")
```

The simple form will search the page by text and default identifier which should be CSS:

```
find("#by_css")
```

This default can be changed with `Capybara.default_selector`:

```
Capybara.default_selector = :xpath
```

Other finder variants are simply more semantic, but they all use `find` under the hood. Then there are `first` and `all` finders:

```
first("a", text: "Home")
first("#menu li", visible: true)
all(:css, "a#person_123")
all(:xpath, '//*[@id="person_123"]')
```

All finders return an element you can query, and `all` returns a collection of elements.

You can also notice we can pass some options to finders, the most useful ones being `visible` and `text` to match on visibility and text of the element.

8.4. Matchers

Matchers are Capybara's assertions and helpers that match elements on the page. Similarly to finders, you pass CSS or XPath selectors, but the element is not returned:

```
# assertion
assert_title "Page title"
assert_text "Any text on page"

# question
assert has_title?("Page title")
assert has_text?("Any text on page")
```

Instead, we either directly assert the condition or get a truthful value from a similar question helper.

We can check for anything on the page using CSS or XPath selectors with `assert_selector` and

has_selector?:

```
assert_selector("p#foo")
assert_selector(:xpath, '//*[@id="foo"]')
assert_selector(:foo)
assert_selector("li", text: "Horse", visible: true)

has_selector?(".highlighted")
has_selector?("p.foo", count: 4)
```

Again CSS is used by default like with finders and we can use similar options like `text` for matching text and `visible` for visibility.

Links and buttons come with specific semantic helpers that also allow us to check against the `href` element or text:

```
has_link?("contact_link", href: "/contact")
has_button?("submit_button", text: "Submit")
```

Similarly, form elements can be checked with `has_field?` that comes with helpful `type` and `with` options:

```
has_field?("Password")
has_field?("phone_number", type: "tel")
has_field?("username", with: "testuser")
```

This helper will match the ID, name, or label.

Checkboxes and selects also get their variants:

```
has_checked_field?("I agree to the terms of service")
has_checked_field?("subscribe_newsletter")
has_select?("Country")
has_select?("country_select", with_options: ["USA", "Canada", "Mexico"])
```

We also get one simple matcher for tables:

```
has_table?(".data-table")
```

As with almost all matches, there are also the direct opposites:

```
assert_no_selector("div.non_existent_class")
```

```
has_no_button?("Submit")
has_unchecked_field?("I agree")
has_no_element?("div.content")
```

We'll see a bit later that it's important to use these negative helpers instead of relying on `assert_not`.

All of these matches are matching directly against the page, but there are also similar matches for preselected elements:

```
element = find("h1")

assert_matches_selector element, "h1"

element.matches_selector?("p.notice:first-of-type")
element.matches_css?("#dynamic-content")
element.matches_xpath?("(/p[@class='notice'])[1]")
element.matches_style?("color" => "rgb(0,0,255)", "font-size" => /px/ )
```

There are quite a few more matchers available and you can find them at the end of the book in the Reference chapter.

8.5. Sessions

Similarly to controller tests, most of our interactions will likely happen for signed-in users. Since this action takes even longer with a browser, we should depend on a signing helper to sign the user for us.

That doesn't mean we wouldn't test the user sign-in flow, but we do it only once and depend on a signing helper for the rest of the test suite.

A typical sign-in flow might look like this:

```
require "application_system_test_case"

class LoginTest < ApplicationSystemTestCase
  setup do
    @user = users(:john)
  end

  test "users can log in" do
    login_as(@user)

    assert has_button?("Sign out")
  end

  private
end
```

```

def login_as(user)
  visit new_user_session_path
  fill_in "E-mail", with: user.email
  fill_in "Password", with: "password"

  click_button "Sign in"
end
end

```

Now depending on your application a sign-in flow might be more complex. Here's a small example of testing OTP code for 2-factor-authentication:

```

...
class LoginTest < ApplicationSystemTestCase
  setup do
    @user = users(:john)
    @user.enable_otp!
  end
  ...
  def login_as(user)
    visit new_user_session_path
    fill_in "E-mail", with: user.email
    fill_in "Password", with: "password"

    click_button "Sign in"

    code = ROTP::TOTP.new(user.otp_auth_secret).at(Time.now)
    fill_in "OTP code:", with: code

    click_on "Submit"
  end
end
end

```

We can also see how we can extract useful repeating patterns to its reusable methods. At the end of the chapter, we'll also see how to organize this to page objects.

Nevertheless, the method takes valuable time and so for other pages, we should depend on a method that can bypass the physical login screen. Here's a small example for those depending on the Devise library.

```

# test/test_helper.rb
class ActiveSupport::TestCase
  ...

```

```
# Devise helpers
include Warden::Test::Helpers
Warden.test_mode!

teardown do
  Warden.test_reset!
end

def log_in(user, opts = {})
  login_as(user, opts)
end

def log_out(*scopes)
  logout(*scopes)
end
end
```

8.6. Timeouts

When we write a test with Capybara, we might request an action that takes a bit longer to manifest.

Imagine a form submit that updates a record:

```
fill_in :data, with: "foo bar"
click_on "Update"

# test the expectation
assert has_content?("Success")
```

In this case, Capybara has us covered. `has_content?` will wait.

What about if we want to assert the opposite?

```
assert_not has_content?("Failure")
```

This code won't work as expected, unfortunately. A natural quick fix could be using `sleep`:

```
# ...

sleep 0.5
assert_not has_content?("Failure")
```

Similarly, the same issue is true for other assertions:


```
assert_not has_css?(".flash")
```

The fix is mostly easy. It's important to use the Capybara alternatives for asserting the negative:

```
# will wait
assert has_no_content?("Success")

# will wait
assert has_no_css?(".flash")
```

In a similar fashion, we should rely on `has_no_field?`, `has_no_link?`, `has_no_table?`, and other Capybara negative assertions.

We can also adjust the maximum waiting time with `default_max_wait_time`:

```
Capybara.default_max_wait_time = ENV.fetch("MAX_WAIT_TIME_IN_SECONDS", 2).to_i
```

If we need to increase this number just for one test, there is `Capybara#using_wait_time` that takes the number of seconds:

```
# in a test

using_wait_time(5) do
  assert page.has_no_css?(".flash")
end
```

If we need to use sleep with something else than Capybara helpers, we can write a helper sleeping up to a predefined wait time:

```
class ApplicationSystemTestCase < ActionDispatch::SystemTestCase
  ...

  def wait_until(time: Capybara.default_max_wait_time)
    Timeout.timeout(time) do
      until value = yield
        sleep(0.1)
      end
      value
    end
  end

  # in test
```

```

wait_until(time: 2.5) do
  current_path == users_path
end

# or

click_on t("devise.registrations.save_changes")
user = wait_until { User.find(@user.id) }

```

We cannot avoid sleeping in Rails system tests, but we should let Capybara do the heavy lifting and skip polluting our tests with random `sleep` calls. Also, `sleep` is a tool. If you open a Capybara test suite, you'll find ad-hoc sleep calls.

8.7. Screenshots

Since we are running system tests with browsers, we can get screenshots on failures. Rails system tests we'll take them automatically for the last visible screen when a test fails. But sometimes it's helpful to make more screenshots, perhaps even on a successful run.

We can take screenshots in Capybara with `take_screenshot` and `take_failed_screenshot`:

```

require "application_system_test_case"

class ProjectsTest < ApplicationSystemTestCase
  ...

  test "listing projects" do
    visit home_path
    take_failed_screenshot
    click_on "Projects"
    take_screenshot
    assert page.has_selector?("h1", text: "Projects")
  end
end

```

Invoking `take_screenshot` will make a screenshot every time it's run while `take_failed_screenshot` only takes a screenshot when the test example fails.

The location for saving the screenshots can be changed by setting `Capybara.save_path`:

```

# test/application_system_test_case.rb
...
Capybara.save_path = Rails.root.join("tmp/capybara")

```

TIP

It might also be a good idea to simply rerun a test case with regular non-headless browser to see how things really look like.

Chapter 9. Continuous Integration

Continuous Integration (CI) is a quality assurance process to integrate frequently recent code changes. Typically we run the full test suite on every pull request commit and then once again after a successful merge.

There are different ways how to go about it. Rails can run the whole test suite with `bin/rails test:all`, but I found in practice I prefer splitting the test suite into two runs. The first part is about the fast models and controllers tests run with `/bin/rails test`. The second part is the often slow system tests.

Specific CI systems and their settings are out of scope for this book, but let's have a look at how GitHub Actions settings could look on a typical Rails application.

To add the test workflow we add a `.github/workflows/test.yml` file for automatic CI runs. First of all, we name the workflow and define when it should run. Let's test on every push to any branch and on a pull request to our master branch:

```
# .github/workflows/test.yml
name: Test and Deploy

# Run for every push or PR
on:
  push:
    branches:
      - '*'
  pull_request:
    branches:
      - main
      - master
```

Our test job might follow after scanning and linting tasks. We define the OS image and any services needed, in this case, a PostgreSQL database and Redis:

```
# .github/workflows/test.yml
...
jobs:
  scan:
    name: Scan for vulnerabilities
    ...
  lint:
    name: Lint code for styling
    ...
  test:
    name: Test
    runs-on: ubuntu-20.04
```

```

services:
  postgres:
    image: postgres:14.2
    env:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: template_test
    ports: ["54320:5432"]
  redis:
    image: redis
    ports: ["63790:6379"]

```

We continue with a sequence of steps that check out the code and prepare all other dependencies:

```

# .github/workflows/test.yml
...
steps:
  - name: Checkout code
    uses: actions/checkout@v4
  - name: Install system packages
    run: |
      sudo apt-get update -qq && sudo apt-get install libvips-dev
  - name: Install Chrome
    id: setup-chrome
    uses: browser-actions/setup-chrome@v1
    with:
      chrome-version: 130
      install-dependencies: true
      install-chromedriver: true

```

The important step here being the installation of exact version of Chrome. This will help us match the version we are using locally and let us install it together with a driver that could otherwise be missing from a standard Ubuntu installation.

We finish it up by installing Ruby and Node dependencies and preparing the test database with `bin/rails db:setup`:

```

# .github/workflows/test.yml
...
steps:
  # ...
  - name: Setup Ruby and install gems
    uses: ruby/setup-ruby@v1
    with:
      ruby-version: .ruby-version

```

```

    bundler-cache: true
  - name: Setup Node
    uses: actions/setup-node@v3
    with:
      node-version: 20.4.0
      cache: "yarn"
  - name: Install packages
    run: |
      yarn install --pure-lockfile
  - name: Setup test database
    env:
      RAILS_ENV: test
    run: |
      bin/rails db:setup

```

Remember that these steps depend on the app you are about to test.

Once all is in place, we can add the steps that run the test suite:

```

# .github/workflows/test.yml
...
  - name: Run tests
    run: bin/rails test
  - name: Run system tests
    env:
      DISABLE_BOOTSNAPE: true
      PARALLEL_WORKERS: 1
      CHROME_BIN: ${{ steps.setup-chrome.outputs.chrome-path }}
      DRIVER_PATH: ${{ steps.setup-chrome.outputs.chromedriver-path }}
    run: |
      RAILS_ENV=test bin/rails assets:precompile
      bin/rails test:system
  - name: Keep screenshots from failed system tests
    uses: actions/upload-artifact@v4
    if: failure()
    with:
      name: screenshots
      path: tmp/capybara
      if-no-files-found: ignore

```

We split the test suite into two to get faster and more reliable results.

For the system tests, we set the right version of Chrome by providing `CHROME_BIN` and `DRIVER_PATH` from the previous `setup-chrome` step. We also set `PARALLEL_WORKERS` for the system test to `1` to disable parallelization.

Finally, we use the `actions/upload-artifact@v4` pre-defined action to get to the Capybara screenshots from the failed system tests. This will let us always download the screenshots of the failed tests from the test run overview.

Once this CI settings is added to the codebase, it should start working automatically after pushing the code to GitHub.

If your system takes a significant time I would also suggest failing fast on a first fail as it often happens that several fails have the same underlying culprit. You can add the `--fail-fast` option to the `run` command above.

Chapter 10. Reference

10.1. Finders

Finders	Examples
<code>find</code> Options: <code>normalize_ws</code> <code>visible</code>	<pre>find("#foo").find(".bar") find(:xpath, '.*div[contains(., "bar")]') find("li", text: "Quox").click_link("Delete")</pre>
<code>ancestor</code>	<pre>element.ancestor("#foo").find(".bar") element.ancestor(:xpath, '.*div[contains(., "bar")]') element.ancestor("ul", text: "Quox").click_link("Delete")</pre>
<code>sibling</code>	<pre>element.sibling("#foo").find(".bar") element.sibling(:xpath, '.*div[contains(., "bar")]') element.sibling("ul", text: "Quox").click_link("Delete")</pre>
<code>find_field</code> - find a form field	<pre># Find a select box by its ID find_field("country") # by name attribute find_field("user[bio]") # by placeholder find_field(placeholder: "Enter your email") # by CSS find_field(css: "#name")</pre>

Finders	Examples
find_link	<pre> find_link("Next") find_link("Records") # Find a link by its text link = find_link("Home") # Find a link by its title link = find_link(title: "About Us") # Find a link by its href attribute link = find_link(href: "/contact") # Find a link by its ID link = find_link(id: "help-link") # Find a link using a CSS selector link = find_link(css: ".nav-link", text: "Contact") </pre>
find_button	<pre> # find by text or value find_button("Submit") find_button("Save") # find by ID or CSS find_button(id: "second-button") find_button(css: ".btn", text: "First Button") </pre>
find_by_id	<pre> find_by_id("user") </pre>
all	<pre> # find by element all("a", text: "Home") all("#menu li", visible: true) # find by CSS or XPath all(:css, "a#person_123") all(:xpath, ".//a[@id='person_123']") </pre>
first	<pre> first(".btn") first("li") </pre>

10.2. Matchers

Matchers either match against an implicit `page` object or explicitly mentioned element.

Document	Examples
<code>assert_title</code>	<pre>assert_title "Page title" assert_no_title "Page heading"</pre>
<code>has_title?</code>	<pre>has_title?("Page title") has_no_title?("Page title")</pre>
<code>assert_selector</code>	<pre>assert_selector("p#foo") assert_selector(:xpath, '//*[@id="foo"]') assert_selector(:foo) assert_selector("li", text: "Horse", visible: true) # XPath expression assert_selector(:xpath, XPath.descendant(:p)) assert_selector("p#foo", count: 4) assert_selector("p#foo", maximum: 10) assert_selector("p#foo", minimum: 1) assert_selector("p#foo", between: 1..10) has_no_selector("p#foo")</pre>
<code>has_selector?</code>	<pre># Check if an element with a specific CSS selector is present has_selector?("div.content") # Check if an element with a specific XPath selector is present has_selector?(:xpath, "//*[@class='content']") # Check if an element with a specific ID is present has_selector?("#main_content") # Check if an element with a specific class is present has_selector?(".highlighted") # Check if an element with a specific text is present has_selector?("p", text: "Welcome to the site")</pre>

Document	Examples
<code>matches_style?</code>	<pre>element.matches_style?('color' => 'rgb(0,0,255)', 'font-size' => /px/)</pre>
<code>assert_matches_styles</code>	<pre>element.assert_matches_style('color' => 'rgb(0,0,255)', 'font-size' => /px/)</pre>
<code>assert_all_of_selectors</code>	<pre>assert_all_of_selectors(:custom, 'Tom', 'Joe', visible: all) assert_all_of_selectors(:css, '#my_div', 'a.not_clicked')</pre>
<code>assert_none_of_selectors</code>	<pre>assert_none_of_selectors(:custom, 'Tom', 'Joe', visible: all) page.assert_none_of_selectors(:css, '#my_div', 'a.not_clicked')</pre>
<code>assert_any_of_selectors</code>	<pre>assert_any_of_selectors(:custom, 'Tom', 'Joe', visible: all) assert_any_of_selectors(:css, '#my_div', 'a.not_clicked')</pre>
<code>assert_no_selector</code>	<pre>assert_no_selector("div.non_existent_class") # Check that no element with a specific ID is present assert_no_selector("#non_existent_id") # Check that no element with a specific class is present assert_no_selector(".hidden") # Check that no element with specific text is present assert_no_selector("p", text: "This text should not be here")</pre>

Document	Examples
has_xpath?	<pre> # Check if an element with a specific XPath selector is present has_xpath?("//div[@class='content']") # Check if an element with a specific XPath and text is present has_xpath?("//p[text()='Welcome to the site']") # Negative test: check that an element with a non- existent XPath selector is not present has_xpath?("//div[@class='non_existent_class']") has_no_xpath? </pre>
has_css?	<pre> # Check if an element with a specific CSS selector is present has_css?("div.content") # Check if an element with a specific CSS class is present has_css?(".highlighted") # Check if an element with a specific CSS ID is present has_css?("#main_content") # Check if an element with a specific CSS attribute is present has_css?("input[type='text']") # Check if an element with specific CSS content is present has_css?("p", text: "Welcome to the site") has_no_css?("div.content") </pre>
has_element?	<pre> has_element?("div.content") has_no_element?("div.content") </pre>

Document	Examples
has_link?	<pre> # Check if there is a link with a specific text has_link?("Home") # Check if there is a link with a specific URL has_link?("About Us", href: "/about") # Check if there is a link with a specific ID has_link?("contact_link", href: "/contact") # Check if there is a link with a specific class has_link?("Support", class: "support-link") has_no_link?("Home") </pre>
has_button?	<pre> # Check if there is a button with a specific ID has_button?("submit_button") # Check if there is a button with a specific name has_button?("Submit") # Check if there is a button with a specific label has_button?("Log In") # Check if there is a button with a specific type has_button?("Save", type: "submit") # Check if there is a button with a specific value has_button?("submit_button", text: "Submit") # Opposite has_no_button?("Submit") </pre>

Document	Examples
has_field?	<pre># Check if there is an input field with a specific ID has_field?("username") # Check if there is an input field with a specific name has_field?("email") # Check if there is an input field with a specific label has_field?("Password") # Check if there is an input field with a specific type has_field?("phone_number", type: "tel") # Check if there is an input field with a specific value has_field?("username", with: "testuser")</pre>
has_checked_field?	<pre># Check if a specific checkbox or radio button is checked by ID has_checked_field?("terms_of_service") # Check if a specific checkbox or radio button is checked by name has_checked_field?("subscribe_newsletter") # Check if a specific checkbox or radio button is checked by label has_checked_field?("I agree to the terms of service") # Check if a specific checkbox or radio button is checked by ID and value has_checked_field?("gender_female", checked: true)</pre>
has_unchecked_field?	<pre>has_unchecked_field?("I agree")</pre>

Document	Examples
has_select?	<pre> # Check if the page has a select field with a specific ID has_select?("country_select") # Check if the page has a select field with a specific name has_select?("user_country") # Check if the page has a select field with a specific label assert has_select?("Country") # Check if the page has a select field with specific options has_select?("country_select", with_options: ["USA", "Canada", "Mexico"]) </pre>
has_table?	<pre> # Check if the page has a table with a specific ID has_table?("#users_table") # Check if the page has a table with a specific class has_table?(".data-table") # Check if the page has a table with specific text content has_table?("Users List") </pre>
assert_matches_selector	<pre> element = find("h1") assert_matches_selector element, "h1" element = find("p.notice") assert_matches_selector element, "p.notice" </pre>
matches_selector?	<pre> element.matches_selector?("#dynamic-content") element.matches_selector?("p.notice") element.matches_selector?("p.notice:first-of-type") element.matches_selector?(:xpath, "//h1") element.matches_selector?(:xpath, "//p[@class='notice']") </pre>

Document	Examples
<code>matches_xpath?</code>	<pre> element.matches_xpath?("//h1") element.matches_xpath?("//p[@class='notice']") element.matches_xpath?("(/p[@class='notice'])[1]") element.matches_xpath?("//div[@id='dynamic-content']") </pre>
<code>matches_css?</code>	<pre> element.matches_css?("#dynamic-content") element.matches_css?("a.next") element.matches_css?("p.notice:first-of-type") </pre>
<code>not_matches_selector?</code>	<pre> assert_no_selector "div#non-existent-element" assert_no_selector "div#dynamic-content", visible: true assert_no_selector "p", text: "This text should not be present" </pre>
<code>assert_text</code>	<pre> assert_text "User will be deleted" </pre>
<code>has_text?</code>	<pre> has_text?("Welcome!") </pre>

Chapter 11. Fin

You've made it!

Going with a default Rails stack will free you from heavy DSLs, long test setup, and slow tests. The minimalistic approach to testing with Minitest and fixtures might not be for everyone, but I hope it inspired you.

Remember that tests are here to help you and to move faster. Focus on what's important.

If you liked Test Driving Rails, you might also like [Kamal Handbook](#), my book on Kamal deploy tool that's now also part of Rails default omakase stack.

A special thank you goes to the book's early supporters and Steven.

Thank you for reading.